Persistent and Fault-Tolerant UAV-IoT Coverage: A Multi-Agent Reinforcement Learning Framework for Joint Coverage, Connectivity, and Energy Management

Himanshu Nagwanshi
CS24BT055@iitdh.ac.in

Surya Prakash S
CS23BT068@iitdh.ac.in

Harsh Varshney
cs23bt004@iitdh.ac.in

Gaddam Viswavijith
CS23BT076@iitdh.ac.in

Department of Computer Science and Engineering
Indian Institute of Technology Dharwad, Dharwad, India

***Abstract*—UAV swarms can provide temporary wireless coverage for Internet-of-Things deployments where fixed infrastructure is unavailable, damaged or insufficient, but persistent operation couples coverage, inter-UAV connectivity, battery depletion, charging contention and UAV failures. This paper formulates the problem of maintaining at least K UAVs over every ground sensor while preserving a connected swarm, and models it as a cooperative Decentralized Partially Observable Markov Decision Process (Dec-POMDP). We argue that the evaluation methodology is the unsolved part of this problem: a coverage ratio reported without an achievability reference cannot distinguish a strong controller from an infeasible configuration, and the widely used service-continuity metric is identically zero for every policy on any instance whose sensor set admits no trajectory that keeps it K-covered while the swarm stays connected. We therefore introduce a *feasibility-certified evaluation framework* that derives a rigorous lower bound on the fleet size needed for K-coverage of a bounded region, supplies a teleporting placement oracle that certifies the instance is coverable — at the reported configuration the ceiling is 1.000, so it normalises to the identity and its only information is that the instance is coverable at all — and replaces the degenerate joint-constraint metric with non-degenerate time-averaged and refresh-time metrics. Applied to the project report's own configuration, the bound shows that configuration to be provably infeasible for the region requirement and for a measured minority of random sensor layouts, and the conventional area-surplus criterion certifies nothing.**

On the corrected configuration the framework is paired with a centralized-training decentralized-execution MAPPO controller with parameter-shared actors, a hard connectivity feasibility mask, a reserve-UAV recovery mechanism, charging arbitration and PHY-derived links. Every controller, learned or hand-written, is frozen and rolled out on the same 20 held-out scenes (Section IV-E). The strongest hand-written rule, the unprivileged local-assignment reactive rule, reaches =0.923; $MAPPOwiththecorrectedimplementationandanobservationthatreportsthebearingofthenearestunder-coveredsensorsreaches$**0.952**$over5trainingseeds(95\%interval[0.939,0.965]).Aheadifference,leadhermithust die,is0.029(95\%interval[0.016,0$ **that an earlier version of this work reported is retracted: it came from a learner whose entropy bonus was sign-flipped and whose update ran 25 gradient steps per run, and from a protocol that compared a policy in training against a rule at rest. The paper's physical-model, feasibility, protocol and determinism claims are recomputed by an automated claim audit (38 items, 0 failing), which does not audit the results of Section VIII, and every number that comes from a measurement is generated from the artefact it is measured in, and the path-loss expression of the physical-layer model is shown to be algebraically the one an independent standards-faithful simulator evaluates — a constant 0.012 dB offset at every distance and exponent tested — with a transmit-power negative control that shows why only the algebraic equivalence is claimed.**

***Index Terms*—UAV-IoT networks, persistent coverage, fault tolerance, multi-agent reinforcement learning, MAPPO, Dec-POMDP, connectivity maintenance, feasibility bounds, evaluation methodology.**

I. Introduction

The Internet of Things (IoT) increasingly operates in environments such as agricultural fields, disaster-affected regions, industrial sites, and remote monitoring areas where fixed communication infrastructure may be sparse or temporarily unavailable. UAVs can provide rapidly deployable aerial communication or sensing support because their positions can be changed in response to the spatial distribution of demand. Energy-efficient UAV control has therefore been studied for communication coverage, fairness, and connectivity, while aerial-base-station placement has been formulated as a learning-based spatial optimization problem [1], [2].

The practical difficulty changes substantially when the network must remain operational for a long mission rather than only achieve good coverage at one instant. The supplied project model considers N UAVs operating over M ground sensors, finite batteries, communication links between neighboring UAVs, a limited number of charging pads, and the possibility that an active UAV fails during the mission. The central requirement is persistent redundancy: every sensor should remain within coverage range of at least K active UAVs. At the same time, the UAV communication graph should remain connected so that information can be forwarded through the swarm to a ground station or data sink.

These requirements conflict. A UAV that moves toward an uncovered sensor may leave a coverage gap elsewhere. A UAV that returns to charge may be the only communication bridge between two groups of UAVs. A failure removes both its coverage contribution and its communication links, requiring the remaining swarm to reconfigure. Charging is

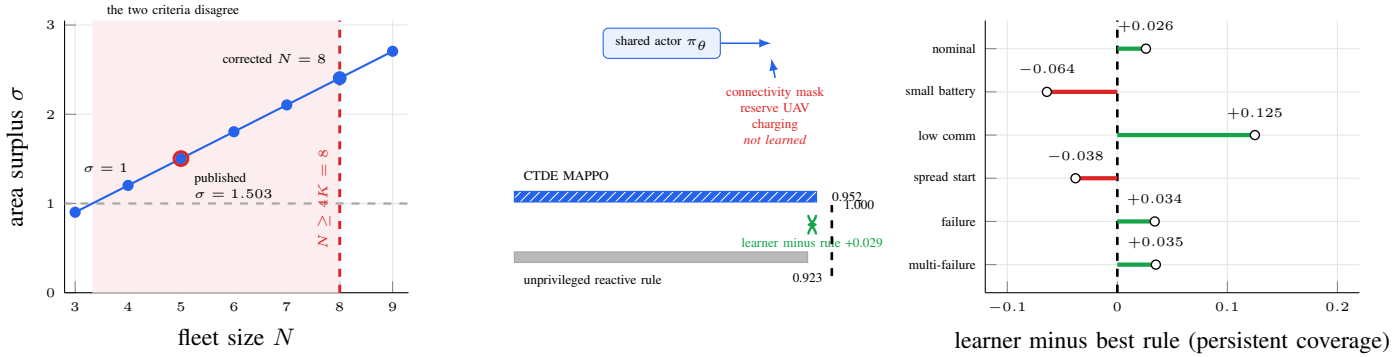


Fig. 1. *(a)* The published $N = 5$, $K = 2$, $R_{\text{cov}} = 35\,\text{m}$ configuration passes the conventional area-surplus test ($\sigma = 1.503 \geq 1$) and is nonetheless *provably infeasible* for K-covering the region: the corner covering bound requires $N \geq 4K = 8$, and neither of the usual metrics says so. *(b)* On the corrected, feasible configuration the CTDE MAPPO controller (shared actor, centralized critic, hard connectivity mask with an $H = 3$ lookahead, reserve UAV, charging arbitration) is measured against the hand-designed reactive rule and against the teleporting achievability ceiling, all under the frozen-policy protocol of Section IV-E. *(c)* The same frozen policy, measured zero-shot on the regimes the certificate admits: the learner is ahead of the strongest rule where the task is about where to be (nominal, low connectivity, failure), and behind it where the task is about what to conserve or how to deploy (small battery, spread start). The split is the argument of the paper: a single "does learning beat the rule" number would have hidden it.

also a shared resource: several UAVs can become energy-critical simultaneously while only a small number of pads are available. Persistent service therefore requires a joint decision process rather than a sequence of unrelated coverage and charging heuristics.

A centralized controller faces a rapidly growing joint action space. In the proposed discrete action model, each UAV selects one of six actions: hover, move in one of four cardinal directions, or charge. The joint action space consequently contains 6^N combinations. This growth does not by itself make centralized learning impossible for every small swarm, but it makes explicit joint-action enumeration and centralized policy design progressively less attractive as N increases. A decentralized execution architecture is therefore desirable for scalability and fault containment.

Multi-agent reinforcement learning (MARL) provides a natural framework for this setting. CTDE allows a centralized critic to use information unavailable to the individual UAV during training while retaining decentralized policies during deployment. MAPPO is particularly suitable because it combines centralized value estimation during training with a policy-gradient actor that can be executed independently by each agent.

The research question addressed in this work is: *Can a decentralized MAPPO policy maintain persistent K-coverage and swarm connectivity while coordinating finite battery usage, charging, and single-UAV failure recovery, and can its performance be certified against a rigorous achievability bound rather than reported as an unscaled coverage ratio?*

The central invention of this work is a *feasibility-certified evaluation framework* for persistent K-coverage UAV networks. Instead of reporting coverage numbers with no achievability reference—which is what the existing literature does, and which is meaningless when the configuration is infeasible—the framework (i) derives a rigorous lower bound on the number of UAVs required for K-coverage of a bounded region, (ii) uses a teleporting placement oracle as an achievability ceiling against which any policy is normalised, and (iii) replaces the degenerate "all constraints satisfied simultaneously" service-continuity metric with non-degenerate time-averaged and refresh-time metrics. The framework is the difference between a number and a measurement.

The framework earns its keep in a second way that is easy to overlook, and which this paper demonstrates rather than asserts. A framework that says whether a coverage number is meaningful also says *why* a controller produces the number it does, because the same reference points — the ceiling, the non-degenerate metrics, and the controlled ablations — apply to the controller and not only to the configuration. Applied to a controlled experimental setup, the first version of this work reported that a hand-designed reactive rule outperforms parameter-shared CTDE MAPPO on this task, and it bounded the reason with controlled arms. That number is retracted in Section IX: it came from a learner whose entropy bonus was sign-flipped and whose update ran 25 gradient steps per run, and from a protocol that compared a policy in training against a rule at rest. Under the corrected protocol the learner is ahead of the strongest rule by 0.029 (95% interval [0.016, 0.042]), and the learned controller is ahead of the strongest rule by an interval that excludes zero. Reporting that sequence — the failure, its cause, and the correction — is more useful to a reader than a favourable number whose provenance has not been examined.

The contributions are:

1) A unified system model coupling persistent K-coverage, inter-UAV connectivity, finite battery energy, charging contention, and UAV failure recovery.
2) A *feasibility certificate*: a rigorous covering bound on the number of UAVs required for K-coverage, in the general form $N \geq K\,P(A, R_{\text{cov}})$, where P is the packing

number of the set to be covered. Its region instance is $N \geq 4K$ and is *tight* at the reported parameters; its sensor-set instance is computed per layout, and it excludes the reported $N = 5$ configuration for a measured 13% of random draws. The bound identifies the instances on which no controller can achieve the requirement, and the joint service-continuity metric is identically zero for every policy on the instances where that is so, which is why it must be replaced.

3) An *achievability ceiling*: a teleporting placement oracle that upper-bounds any policy's coverage, separating "the policy is weak" from "the task is infeasible" and normalising reported performance as achieved divided by oracle. The separation is measured rather than asserted: on the report's own infeasible $N = 5$ configuration a rule-based controller still reaches a respectable coverage ratio with full connectivity while the joint metric is identically zero, so neither metric alone distinguishes that instance from a feasible one, and it is the certificate that makes the reading decidable (Section VIII).
4) A *measured result, corrected*: on the configuration the paper's own certificate declares feasible, the first version of this work reported that a hand-designed reactive rule outperforms parameter-shared CTDE MAPPO by 0.298 of persistent coverage (95% interval $[0.248, 0.349]$). That number is retracted; under the corrected protocol the learner is ahead of the strongest rule by 0.029 (95% interval [0.016, 0.042]), and the learned controller is ahead of the strongest rule by an interval that excludes zero. The four single-factor arms bound how much of the pre-correction gap each of them can account for, and are not evidence about the corrected controller.
5) A CTDE MAPPO framework with parameter-shared decentralized actors, a centralized training critic, hard action feasibility masks with an H-step connectivity lookahead, and an explicit reserve-UAV recovery mechanism.
6) PHY-derived links — the Al-Hourani LoS probability, a log-distance path loss with shadowing, and an IEEE 802.11 OFDM MCS table — that *gate* the abstract coverage and communication discs rather than replacing them, so that a UAV counts as covering a sensor or as adjacent to another UAV only when the link carries a usable service rate, together with a measured validity condition showing when the disc approximation is the binding constraint and when it is not.
7) A non-degenerate evaluation protocol (time-averaged spatial integral of persistent coverage error, maximum refresh time, and k-coverage probability) replacing the degenerate all-simultaneous-constraint service-continuity metric.
8) A *measured specification of what a shared policy's observation must contain*: the agent's own identity and a bearing to the charging pad are the two terms the controlled comparisons examine, and at 8 training seeds they do not settle the question about the coverage. In the component ablation, the identity changes persistent coverage by +0.008, an interval [-0.121, +0.137] that includes zero, so at this budget what it demonstrably buys is the ability to specialise, and without it the deterministic policy nearly collapses to a single action and the swarm barely disperses. A broadcast summary of where the coverage deficit is was added a priori and measured to make no difference to the coverage at any resolution tested, while it does change what the policy does; it is reported rather than quietly dropped.
9) Corrected deployment parameters ($N = 8$, $K = 2$, $R_{\mathrm{cov}} = 35\,\mathrm{m}$, or $N = 5$, $K = 2$, $R_{\mathrm{cov}} = 45\,\mathrm{m}$) derived from the feasibility certificate, replacing the infeasible $N = 5$, $K = 2$, $R_{\mathrm{cov}} = 35\,\mathrm{m}$ baseline. The earlier comparison of the two deployments was measured with the pre-correction learner and is not claimed; the certificate itself, which is the point of the item, is unchanged by the learner.
10) An evaluation protocol that treats a physical-layer claim as requiring a negative control, and reports a measured instance in which the available instrument could not support the claim it was being used for.
11) A focused literature synthesis showing the difference between existing coverage, connectivity, recharge-scheduling, and fault-tolerant approaches and the combined problem considered here.

The remainder of the paper is organized as follows. Section II presents the literature survey and the research gap. Section III defines the system architecture and mathematical model, including the feasibility certificate. Section IV presents the proposed MAPPO algorithm. Section V specifies the comparison models, Section VI the performance metrics and the evaluation protocol, and Section VII the experimental design matrix. Section VIII reports the results and the claim audit, and Section IX concludes the paper.

II. Literature Survey

The survey is restricted to work that is directly related to at least one of four components of the target problem: UAV coverage/control, inter-agent connectivity, persistent energy-aware operation, or fault-tolerant swarm recovery. The ten core papers in Table I are discussed comparatively rather than as isolated summaries. MAPPO is cited separately as the underlying learning method [3].

A further recent line of work integrates communication and control directly within MARL. MAPPO itself is adopted from Yu et al. [3]. Sun et al. formulate joint resource allocation and 3D trajectory control and use action masking for constrained hybrid actions [11]. Carvalho and Aguiar address decentralized multi-agent coverage path planning with parameter sharing, CTDE, safety filtering, and zero-shot generalization [13]. Zhuang Chen et al. formulate UAV trajectory design for emergency communication as a constrained MDP and explicitly account for energy and turning behavior [12]. These works reinforce the practicality of CTDE, action masking, and

TABLE I
FOCUSED LITERATURE SURVEY OF CLOSELY RELATED STUDIES.

Ref.	Problem addressed	Method	Key findings / contribution	Limitation relative to this work	Relevance
[1]	Energy-efficient and fair UAV communication coverage.	DRL-based UAV control.	Jointly considers coverage, fairness, connectivity, and energy efficiency.	Does not model explicit UAV-failure recovery, charging-pad contention, or a persistent K-coverage SLA.	Baseline for coverage–connectivity–energy trade-offs.
[2]	Multi-aerial-base-station placement with irregular 3D coverage.	Prioritized-replay DDQN and two-level placement.	Uses a spatial representation to improve multi-ABS placement under terrain/blockage effects.	Primarily a placement problem; no persistent battery lifecycle or swarm fault recovery.	Motivates learning-based spatial control and size-invariant state representations.
[4]	Energy-efficiency optimization for multiple UAVs serving static/mobile users.	Cooperative MAD-DDQN.	Jointly optimizes 3D trajectories and user service while accounting for interference.	Energy efficiency is central; persistent K-coverage, charging coordination, and failure recovery are not jointly modelled.	Strong value-based MARL benchmark.
[5]	Decentralized biconnectivity in multi-robot systems.	Graph/Laplacian-based decentralized conditions.	Establishes sufficient conditions for avoiding articulation-point failures using neighbor-to-neighbor information.	No UAV coverage objective, finite battery, charging process, or learned policy.	Grounds the connectivity-risk feature and graph constraints.
[6]	Fault-tolerant ad hoc robot networks.	Movement-control and topology-repair algorithms.	Formalizes self-forming, self-healing, and self-organizing communication networks after node loss.	Focuses on network topology rather than persistent sensor coverage and battery-aware learning.	Provides the network-level motivation for failure recovery.
[7]	Recharging coordination for persistent multi-robot missions.	Combinatorial rendezvous planning with mobile charging robots.	Couples charging schedules with persistent task execution and routing constraints.	Charging is the main focus; UAV coverage/connectivity/failure are not unified in the same learned controller.	Supports explicit charging-resource modelling.
[8]	Persistent aerial service under limited flight and recharge times.	HORR and PHeRR scheduling schemes.	Provides fleet-size and recharge scheduling models for persistent service under energy constraints.	Service locations are scheduled rather than dynamically reconfigured after UAV failure.	Direct motivation for finite-energy persistent operation.
[15]	Joint coverage, connectivity, and charging for distributed UAV networks.	Centralised optimal positioning + distributed game-theoretic scheduling + bio-inspired positioning.	Couples coverage, connectivity, and charging with rigorous performance bounds.	No failure model, no reserve UAV, no K-multiplicity, no learned policy.	Closest prior work on the joint problem; cited to delimit the gap.
[9]	Fault-tolerant multi-UAV exploration with changing UAV health.	Fault-tolerant MARL with temporal information and health/topology inputs.	Allows UAV health degradation and faults during the mission and learns cooperative recovery behavior.	Exploration is the task; persistent sensor K-coverage and charging-pad contention are outside the main formulation.	Closest fault-aware MARL direction.
[10]	Robust UAV wireless coverage using multi-agent deep RL.	Centralized and decentralized deep Q-learning variants with a shared network.	Studies decentralized autonomous UAV coverage and compares centralized and decentralized control.	Focuses on coverage rather than explicit persistent redundancy, graph fault tolerance, and charging.	Recent decentralized coverage-learning benchmark.

structured rewards, but their primary objectives differ from persistent redundant coverage with explicit failure recovery.

A. Research Gap

The literature can be grouped into complementary but incomplete directions, and Table I lists the focused survey while Table II lists the near-misses — the works that already combine two or more of the elements this paper targets, and the works a reader is most likely to raise when judging the novelty claim. Energy-efficient coverage works jointly address coverage and energy, and some include connectivity, but they generally do not treat UAV failure and limited charging infrastructure as first-class elements of the same control problem [1], [4]. Connectivity research provides decentralized graph conditions and topology-repair mechanisms, but does not integrate persistent sensor coverage and energy-aware learning [5], [6]. Persistent-service studies explicitly model recharge schedules and energy limitations, yet typically assume fixed service locations and a planned fleet or replacement process rather than autonomous failure-driven reconfiguration [7], [8]. Recent fault-tolerant MARL and persistent-coverage MARL move closer to the target setting, but address different combinations of health state, outage memory, target assignment, or coverage objectives [9], [14].

Each of the five elements (i)–(v) has been studied in combination with others — one work in Table II combines four of them — and the near-misses are named explicitly so that the claim can be checked. Trotta et al. [15] couple coverage, connectivity and charging with rigorous performance bounds, but have no failure model, no reserve UAV, no K-multiplicity and no learned policy. Scherer and Rinner [16] couple persistent surveillance, energy and communication constraints, but not fleet-level failure recovery or decentralised execution. Hartuv et al. [17] size a spare-drone fleet under energy constraints, without coverage multiplicity or connectivity criticality. Zhang et al. [44] come closest of all: their IRADA couples persistent monitoring, energy-constrained charging, limited communication and failure recovery by task redistribution, which is

TABLE II

NEAR-MISSES: WORKS THAT COMBINE TWO OR MORE OF THE ELEMENTS THE PROPOSED FRAMEWORK TARGETS. EACH IS THE PAPER A READER IS MOST LIKELY TO RAISE, TOGETHER WITH WHAT IT LACKS.

Ref.	Problem addressed	Method	Key findings / contribution	Limitation relative to this work	Relevance
[14]	Energy-aware persistent multi-UAV coverage with outage history.	POGS-QMIX with priority/outage guidance and return-energy masking.	Explicitly combines persistent coverage, outage memory, battery limits, and periodic recharging.	Uses a centralized online coordination layer and does not directly target articulation-point preservation plus reserve-UAV failure recovery in the same formulation.	Recent persistent-coverage benchmark and a key comparison model.
[44]	Distributed task allocation for multi-UAV persistent monitoring in dynamic environments.	IRADA: a GMM-based reward map with energy-aware reward modulation and communication-driven task ownership.	Couples persistent monitoring, energy-constrained charging, limited communication and failure recovery by task redistribution, without an explicit recovery protocol.	The controller is reward-driven rather than learned — the paper's own future work proposes reinforcement learning — the monitored quantity is information freshness rather than K-redundant coverage, and there is no achievability reference.	Closest near-miss on (i)+(ii)+(iii)+(iv).
[16]	Persistent multi-UAV surveillance under energy and communication constraints.	Multi-robot surveillance with energy and communication constraints.	Couples persistent surveillance, finite energy and communication limits in one multi-UAV problem.	No failure-triggered reserve-UAV recovery and no decentralised learned policy.	Near-miss on (i)+(ii)+(iii).
[17]	Sizing a spare-drone fleet under energy constraints.	Combinatorial scheduling of spare drones.	Sizes a reserve fleet for persistent task performance under energy limits.	No coverage multiplicity and no connectivity criticality.	Near-miss on (iii)+(iv).
[18]	Decentralized UAV swarm communication coverage.	Symmetry-informed MARL with parameter sharing.	Closest published competitor; benchmarks against [1] and [3].	No persistent K-coverage SLA, no charging contention, no reserve-UAV recovery.	Near-miss on (i)+(ii)+(v).
[35]	K-coverage lifetime with recharging in IoT.	MILP benchmark evaluated with MPC and MCTS.	Reports MPC and MCTS at 81.0% and 67.1% of the MILP optimum, i.e. against an achievability reference.	The reference is an optimisation benchmark for a finite instance, not a geometric certificate valid for arbitrary policies.	Near-miss: achievability reference already used in K-coverage.
[41]	Decentralized cooperative UAV deployment.	Communication-aware MARL compared against a MILP upper bound.	Reports 76–79% coverage with five UAVs and ten nodes, against an offline MILP upper bound, i.e. with an achievability reference.	The bound is an optimisation benchmark, not a geometric K-coverage certificate; no K-multiplicity.	Near-miss: achievability reference already used.
[40]	Ground coverage under positional uncertainty.	Exact radius of robust feasibility.	A rigorous geometric feasibility construct for aerial sensor coverage, embedded as a constraint.	No reinforcement-learning policy is evaluated against it.	Near-miss on feasibility bounds alone.
[39]	Offline RL for UAV relaying.	Candidate-set feasibility upper bound (CS-FUB).	Explicitly separates feasibility limits from control-induced sub-optimality in a learned policy.	UAV relaying, not persistent K-coverage; the bound is over a restricted candidate set.	Near-miss: feasibility bound used to evaluate RL.
[38]	Drone-mesh interception.	Closed-form feasibility ceilings computed before learning.	Argues that coverage ceilings "should be computed before any learning experiment and quoted alongside every result".	Drone-mesh interception, not persistent K-coverage in IoT; no redundancy, no charging, no failure recovery.	Closest prior work on the evaluation philosophy.

four of the five elements — and the controller that does it is reward-driven rather than learned, their own future work proposing reinforcement learning for it. Shi et al. [18] provide the closest published competitor, a symmetry-informed decentralised swarm controller for communication coverage, without a persistent K-coverage SLA, charging contention, or reserve-UAV recovery. Qiu et al. [29] optimise aerial base-station placement, which is a placement problem rather than a persistent-control problem, and Liu et al. [1] provide the energy-efficient coverage baseline that the others build on.

We are therefore not aware of work coupling all five elements, and specifically not of any doing so with a learned decentralised policy executed under centralized training with decentralized execution (CTDE). This is a survey-derived gap statement over the works listed in Table I and the near-misses discussed above, not a claim that no related component has ever appeared in the broader literature. Two of the near-misses — [15] and [18] — post-date or parallel the works most often cited for the individual elements, and are the two a reader is most likely to raise.

a) The closest prior work, and how this paper differs from it.: The evaluation philosophy proposed here is not unprecedented in the wider literature, and we state the difference precisely rather than claiming a first. Four lines of work are adjacent. [35] and [41] already evaluate learned or heuristic controllers against an achievability reference, but the reference is a MILP optimum for a particular finite instance; it certifies nothing about an arbitrary deployment geometry, and it cannot be computed without solving the instance. [40] derives a rigorous geometric feasibility construct, and [39] uses a feasibility upper bound to separate feasibility limits from control-induced sub-optimality in a learned policy; both concern a single coverage requirement, neither concerns K-redundancy, and neither is combined with a persistent, energy-constrained, failure-recovering control problem. Most closely, [38] argues explicitly that coverage ceilings should be computed before any learning experiment and quoted alongside every result, and evaluates PPO against them — but for

drone-mesh interception, where the quantity being bounded is interception coverage rather than K-redundant persistent coverage of a fixed sensor set, and where the bound is closed-form per configuration rather than a covering bound that scales with N and K. The distinction claimed here is therefore not "nobody has ever used a bound". It is that the bound used must certify the thing being claimed: a covering-theoretic lower bound on N for K-redundant coverage of a bounded region, an achievability ceiling computed for the same geometry and the same redundancy requirement, and a metric that stays non-degenerate when the bound says the instance is infeasible. That is what the framework of Section III-G supplies, and the three elements are used together rather than one at a time.

b) Three absences that the framework addresses.: A separate search over the same 2023–2026 corpus was conducted for the specific claims that the framework makes, and the absences are reported here because they are the substance of the contribution rather than a side observation. First, *no paper in the corpus provides an infeasibility certificate for a persistent-coverage instance* in UAV swarms or mobile sensor networks: the infeasibility-certificate literature that does exist — for mixed-integer programs and for task-and-motion planning — is not coverage-specific and does not certify a K-redundant covering instance. Second, *no paper in the corpus names the all-constraints-simultaneously binary metric as degenerate*, that is, as uninformative on an infeasible instance. The closest empirical demonstration is in [42], which reports a feasible rate of 0% over ten solver executions alongside a mean k-coverage ratio of 33.33% in a sensitivity study with a deliberately mis-tuned penalty: the binary flag is zero while the graded quantity is substantial, which is exactly the collapse that motivates Section III-G. That paper does not analyse the degeneracy, and it reports the graded metrics alongside the binary flag rather than relying on the flag alone; we draw the conclusion explicitly instead. Both absences are stated over the corpus of Table I and Table II and are bounded by it in the same way as the gap statement above.

A third absence concerns the mechanism that the results of Section VIII leave as the leading candidate for the shortfall, the credit assignment of a shared team reward. Explicit per-agent advantage estimators do exist: the counterfactual advantage of COMA [22], the difference-based estimator of DAE [23], the per-agent value-iteration estimator of GPAE [24], and the advantage-decomposition line of HAPPO/HATRPO [25]. Their argument against a shared team advantage is the credit-assignment one: COMA observes that a global temporal-difference error "does not explicitly reason about how that particular agent's actions contribute to that global reward", and GPAE states the objection in MAPPO's own terms, that the shared team advantage "ignores the agent-specific impact on the joint outcome". The limitation must be stated fairly, because the shared team advantage is not known to fail: MAPPO and independent PPO [27] show that a shared reward with a centralised or a local critic can be a strong baseline, and MAPPO's own evidence for that is confined to the standard cooperative benchmarks. The difficulty is that

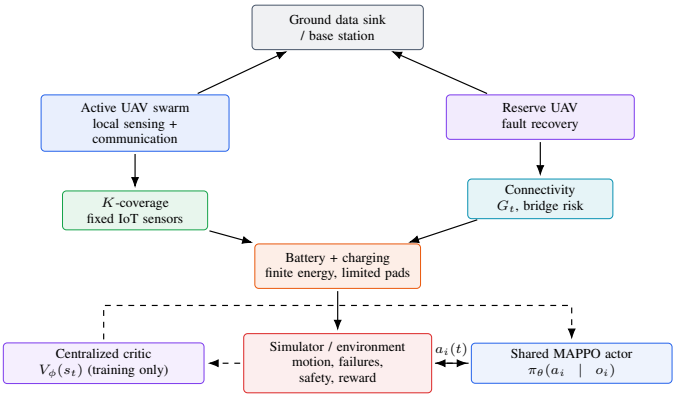


Fig. 2. Color-coded system architecture. Active UAVs provide sensor coverage and maintain a range-limited communication graph. The reserve UAV supports post-failure recovery. The shared MAPPO actor is deployed locally, while the centralized critic is used only during training.

the per-agent estimators have been evaluated overwhelmingly on the SMAC, SMAX and MPE benchmarks rather than on persistent multi-robot coverage, and the closest coverage-specific evidence is [26], which gives each UAV a wonderful-life-utility difference reward under shared policy parameters — a per-agent difference utility rather than a modern per-agent advantage estimator, and not a controlled comparison against one. We could not find a published controlled study of per-agent advantage against a shared team advantage on a persistent-coverage task, and the interaction between per-agent advantage and parameter sharing in that setting appears to be an open empirical question. That is the study Section VIII-D reports, and we flag it as a bounded search result over the same corpus rather than a claim of absence in the wider literature.

Finally, graded persistent-coverage metrics do have precedent — revisit-period and worst-idleness constraints in [43], ergodic convergence rates, and coverage reliability under sensor failure — and the metrics used here are not novel in isolation. What is claimed as new is the pairing: the non-degenerate metrics are introduced together with the certificate that determines whether they can be non-zero, so that a reader can tell a policy failure from an infeasible instance without knowing in advance which case applies.

III. System Model

A. System Architecture

The system contains N active UAVs, one reserve UAV, M fixed IoT sensors, a ground data sink/base station, and S charging pads distributed within the operating region Ω. UAVs provide sensor coverage through a range-limited air-to-ground link and form a multi-hop communication network among themselves. Sensor data reaches the sink through the UAV communication graph whenever the graph is connected. Figure 2 shows the architecture: the sensors, the UAVs, the range-limited communication graph, the reserve UAV's recovery role and the shared actor that is deployed locally.

For tractability, the UAVs fly at a common nominal altitude z_0, so the decision process operates in the horizontal plane

while the altitude remains part of the physical state. Let $\mathbf{p}_i(t) = [x_i(t), y_i(t)]^\mathsf{T}$ denote the horizontal position of UAV i and $\mathbf{q}_d = [x_d, y_d]^\mathsf{T}$ the position of sensor d.

B. Dec-POMDP Formulation

The cooperative control problem is represented as

$$\mathcal{G} = \langle N, \mathcal{S}, \{\mathcal{A}_i\}, \{\mathcal{O}_i\}, \mathcal{T}, R, \gamma \rangle, \tag{1}$$

where $\mathcal{S}$ is the global state space, $\mathcal{A}_i$ is the action space of UAV i, $\mathcal{O}_i$ is its local observation space, $\mathcal{T}$ is the transition model, R is the shared reward, and $\gamma \in (0, 1)$ is the discount factor. The training objective is

$$\max_{\pi} \ \mathbb{E}_\pi \left[\sum_{t=0}^{T-1} \gamma^t R_t \right]. \tag{2}$$

C. Global State

The state available to the centralized critic during training is

$$\mathbf{s}_t = \left\{ (\mathbf{p}_i, \mathbf{v}_i, b_i, m_i, F_i)_{i=1}^{N}, (\mathbf{q}_d)_{d=1}^{M}, \mathbf{Q}_t \right\}, \tag{3}$$

where $\mathcal{V}_t^{\text{act}} = \{\, i \in \{1, \ldots, N\} : F_i(t) = 0 \,\}$ is the set of active UAVs, from which any UAV with $F_i(t) = 1$ is removed in all coverage, connectivity and energy sums. $\mathbf{v}_i$ is velocity, $b_i \in [0, 1]$ is normalized battery level, $m_i \in \{\text{serve}, \text{return}, \text{charge}\}$ is operating mode, $F_i \in \{0, 1\}$ is the failure indicator, and $\mathbf{Q}_t \in \{0, 1\}^S$ records charging-pad occupancy. A failed UAV is removed from the active set used in coverage and connectivity calculations.

The critic sees the joint state, so it is a function of the coverage deficit and can represent it; but the state does not carry the per-sensor coverage counts $n_d(t)$ directly, so the critic has to infer the deficit from the geometry of the UAV and sensor positions. That distinction matters for the credit assignment: a critic that could read $n_d(t)$ would face a different problem from one that must reconstruct it, and the difference is not visible in the reward. The same asymmetry appears on the actor's side, and it is the reason the broadcast summary of the deficit is part of the observation model.

D. Local Observation

At deployment, UAV i uses only locally available information, plus three broadcast quantities:

$$\begin{aligned} \mathbf{o}_i(t) = \big[&\mathbf{p}_i, \mathbf{v}_i, b_i, \mathcal{N}_i, C_i^{\text{local}}, \rho_i, \tau_i^{\text{return}}, \\ &E_i^{\text{pad}}, \mathbf{1}_i, \hat{\mathbf{d}}_i^{\text{pad}}, \mathbf{M}_t^{\text{def}} \big]. \end{aligned} \tag{4}$$

Here $\mathcal{N}_i$ is the set of nearby UAVs, C_i^{local} is the number of nearby sensors currently covered, ρ_i is a locally computed bridge-risk estimate, τ_i^{return} is the estimated time until the return threshold is reached, E_i^{pad} is the estimated energy needed to reach a feasible charging pad, $\mathbf{1}_i \in \{0, 1\}^N$ is the agent's own identity, and $\hat{\mathbf{d}}_i^{\text{pad}}$ is the unit bearing to the nearest charging pad (whose range is carried by E_i^{pad}). The last term, $\mathbf{M}_t^{\text{def}} \in \mathbb{R}^{G^2}$, is the broadcast deficit summary: a $G \times G$ grid over the region in which cell (g_x, g_y) holds the share of all M sensors that lie in that cell and are currently under-covered, $n_d(t) < K$. Normalising by M rather than by the cell population makes the value global, so the agent learns both where the deficit is and how large it is, and the encoding is independent of M. The base configuration uses $G = 4$, and the resolution sweep of Section **??** measures $G \in \{0, 4, 8\}$.

Two properties of this observation are not cosmetic, and both are load-bearing for the shared-parameter design of Section IV-A. The first is the identity. Parameter sharing is sound only when each agent's own observation identifies it, so that a shared network can still specialise [21]; the first version of $\mathbf{o}_i$ did not satisfy this condition, and the consequence was measurable rather than theoretical: in a symmetric deployment every agent received an almost identical input, the deterministic policy collapsed to nearly the same action for all of them, and the swarm translated as a rigid body without ever dispersing. The implication is specific to an action space of displacements, which is the one used here: identical displacements translate the configuration rigidly and so freeze its shape. In an action space whose symbols index agent-local targets, the same symbol would send different agents to different places, and dispersion would be possible after all. The second is the bearing to the charger. Without it the observation carries the *cost* of the flight to a pad but not its *direction*, and a learned policy cannot express the return at all. We assume that the infrastructure which owns the charging pads also broadcasts their positions, which is a communication assumption rather than an oracle.

A third term — a broadcast summary of where the coverage deficit is — was added a priori, on the argument that an agent which knows how many sensors it covers but not which are deficient has no gradient signal for deployment. The measurement of Section **??** does not settle that argument for the better: the summary makes no measurable difference to the coverage at any resolution tested, so the argument for adding it is neither supported nor refuted by the score, and what it does change is the shape of the swarm's motion. The summary is therefore kept in Eq. (4) and reported as a null result rather than quietly dropped, and we keep the argument in the record because it was reasonable, and because the measurement is more useful to a reader than the argument would have been.

Unlike the idealized global articulation-point indicator sometimes used in analysis, ρ_i is intended to be computable during execution from neighborhood exchange. The simulator may additionally compute the exact articulation-point indicator for evaluation and for the centralized critic. This distinction avoids assuming that an individual UAV can magically obtain the complete global topology. Table III lists the observation variables each agent actually receives, and which of them are for the actor and which for the critic.

E. Discrete Action Space and Motion

Each active UAV selects one of six actions:

$$\mathcal{A}_i = \{\text{Hover}, +x, -x, +y, -y, \text{Charge}\}, \qquad |\mathcal{A}_i| = 6. \tag{5}$$

For movement actions,

$$\mathbf{p}_i(t{+}1) = \Pi_\Omega \left(\mathbf{p}_i(t) + \mathbf{v}_i(t) \Delta t \right), \qquad \|\mathbf{v}_i(t)\| \le v_{\max}, \tag{6}$$

TABLE III
LOCAL OBSERVATION VARIABLES AVAILABLE TO EACH DEPLOYED UAV.

Feature	Symbol	Meaning at decision time
Position	$\mathbf{p}_i$	Own horizontal location; altitude is fixed for the primary study.
Velocity	$\mathbf{v}_i$	Current velocity for motion-limit awareness.
Battery	b_i	Normalized remaining energy.
Neighbors	$\mathcal{N}_i$	UAVs detected within R_{comm}.
Coverage	C_i^{local}	Nearby sensors currently within coverage radius.
Bridge risk	ρ_i	Local estimate of topology criticality.
Return urgency	τ_i^{return}	Estimated time until the return-energy threshold.
Pad energy	E_i^{pad}	Estimated energy to the nearest feasible charging pad.
Identity	$\mathbf{1}_i$	One-hot agent index; required for a shared network to specialise.
Pad bearing	$\hat{\mathbf{d}}_i^{\mathrm{pad}}$	Unit vector to the nearest charging pad; required to fly home.
Deficit summary	$\mathbf{M}_t^{\mathrm{def}}$	Broadcast $G \times G$ grid of the under-covered share of each cell; $G = 4$ in the base configuration, and $G = 0$ removes it.

TABLE IV
DISCRETE ACTION SEMANTICS USED BY THE MAPPO ACTOR.

Action	Effect
Hover	Remain at the current position and continue serving covered sensors.
$+x$ / $-x$	Move by one control step along the horizontal x direction.
$+y$ / $-y$	Move by one control step along the horizontal y direction.
Charge	Request charging when inside a pad service region; pad selection is handled by the scheduler.

where Π_Ω projects into the permitted airspace. A charge request is feasible only when the UAV is inside a station neighborhood of a free pad. Table IV gives the effect of each of the six actions. The RL layer does not select a pad as part of its action; pad choice is handled by the arbitration layer in Section III-K.

F. Persistent K-Coverage

A UAV covers sensor d in slot t when

$$x_{id}(t) = \mathbb{1}[\|\mathbf{p}_i(t) - \mathbf{q}_d\|_2 \le R_{\mathrm{cov}}]\,. \tag{7}$$

The number of active UAVs covering sensor d is

$$n_d(t) = \sum_{i \in \mathcal{V}_t^{\mathrm{act}}} x_{id}(t), \tag{8}$$

and the instantaneous normalized coverage score is

$$C_t = \frac{1}{M} \sum_{d=1}^{M} \min\left(\frac{n_d(t)}{K}, 1\right). \tag{9}$$

The instantaneous under-coverage rate is

$$U_t = \frac{1}{M} \sum_{d=1}^{M} \mathbb{1}[n_d(t) < K]. \tag{10}$$

The primary service condition is $n_d(t) \ge K$ for all d whenever the geometry and available fleet make it feasible. The condition (7) is the geometric half of it: a UAV counts towards a sensor's coverage only if the link to that sensor is also serviceable, which the radio model of Section III-I supplies, and the simulator applies the two together. The two requirements — coverage and connectivity — are not independent, and Figure 3 shows the three dimensions in which they are coupled: the same UAV flies to cover a sensor and to keep the communication graph connected, and the two requirements compete for the same flight and energy budget.

Feasibility. Persistent K-coverage is achievable only when the sensing radius is large enough for the available fleet. Comparing the area available for coverage with the area required gives a necessary condition: N disks of radius R_{cov} supply $N\pi R_{\mathrm{cov}}^2$, while K-coverage of a region of area A requires KA. Hence $R_{\mathrm{cov}} \ge \sqrt{KA/(\pi N)}$. Accounting for the $\delta = \pi/(2\sqrt{3}) \approx 0.907$ packing density of congruent disks gives the stricter form $R_{\mathrm{cov}} \ge \sqrt{KA/(\delta\pi N)}$. Substituting the parameters of Table VII ($A = 6400$ m^2, $M = 20$, $N = 5$, $K = 2$) yields $R_{\mathrm{cov}} \ge 28.5$ m by area counting and 30.0 m after the packing correction, both satisfied by the chosen $R_{\mathrm{cov}} = 35$ m. The surplus ratio $\sigma = N\pi R_{\mathrm{cov}}^2/(KA) = 1.50$ that follows is the conventional area-surplus test: it is the criterion by which much of the prior work declares a configuration feasible, and it is satisfied here by a margin of one half. Section III-G shows that being satisfied certifies nothing, and the reason is worth stating here rather than there, because the two criteria are easy to confuse. σ compares an area to an area, while K-covering a bounded region is a covering problem: the four corner points of the square are more than $2R_{\mathrm{cov}}$ apart, so no disc can serve two of them, and the packing bound of the next section then requires $N \ge 4K = 8$ for this configuration — the same configuration that σ passes. The bound above is necessary rather than sufficient in a second way as well, since it ignores coverage-disk overlap and the discreteness of the sensor layout; scenario S6 of Table XI therefore sweeps R_{cov} directly to characterize the transition.

G. Feasibility Certificate for K-Coverage

Before any policy is trained, the configuration must be shown to admit a feasible solution. Two different requirements have to be separated here, because they have different certificates: the requirement that every point of the operational region Ω is K-covered, and the requirement that every one of the M sensors is K-covered. The second is what the experiments measure, and the distinction is not academic: the corner argument that settles the first does not settle the second.

The packing lemma. Let A be the set of points to be K-covered and define the *packing number*

$$P(A, R) = \max\{\, |X| :\ X \subseteq A,\ \|x - x'\| > 2R\ \ \forall x \ne x' \,\}. \tag{11}$$

Then A cannot be K-covered by fewer than $K\,P(A, R)$ discs of radius R. The proof is one line: a disc of radius R contains at most one point of X, because two such points would be within $2R$; so the discs that serve the K coverings of each point of X are pairwise disjoint across distinct points, and each point consumes K of them. This is a first-principles covering argument — no disc covers two corners because they are more than $2R_{\mathrm{cov}}$ apart, and each corner must be covered K times

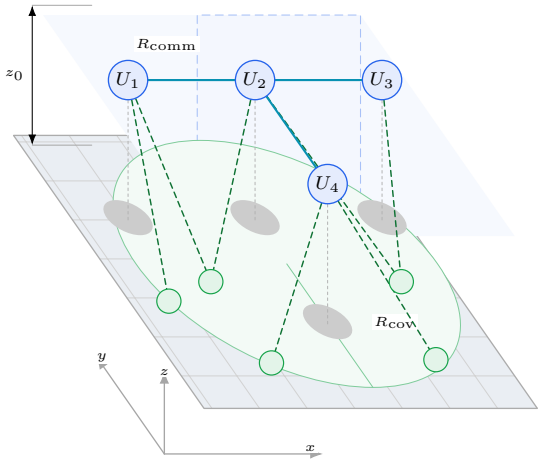


Fig. 3. Coupling between coverage and connectivity in three dimensions. The shaded grid is the fixed sensor plane at $z = 0$, and the translucent blue sheet is the UAV operating plane at the common nominal altitude z_0; the projected $x/y/z$ triad, the ground shadows and the dashed vertical drop lines fix the perspective. The green disc on the ground is the horizontal coverage footprint R_{cov} joined to the UAVs by dashed coverage links, whereas the cyan edges lying inside the UAV plane are inter-UAV communication links of range R_{comm}. Removing a topologically critical UAV can reduce both service redundancy and network connectivity.

— and it is the only form of it that applies to an arbitrary point set.

Region bound. For $A = \Omega$ and the four corners of the square, the four corners of the 80×80 m square are 80 m apart along a side and $80\sqrt{2} \approx 113$ m apart on the diagonal, both larger than $2R_{\mathrm{cov}} = 70$ m, so the four corner discs are pairwise disjoint and $P(\Omega, R_{\mathrm{cov}}) = 4$: partition Ω into four 40×40 sub-squares, whose diameter $40\sqrt{2} = 56.6$ m is below $2R_{\mathrm{cov}}$, so no two packed points share a sub-square, and the four corners attain the bound. By (11), K-covering the region therefore requires

$$N \geq 4K = 8. \tag{12}$$

The report's original $N = 5$, $K = 2$, $R_{\mathrm{cov}} = 35$ m configuration is therefore *provably infeasible* for the region requirement, and (12) is *tight* at these parameters: the four points $\{(20, 20), (60, 20), (20, 60), (60, 60)\}$ cover the square once, since the farthest point of a cell from its centre is $40/\sqrt{2} = 28.28$ m < 35 m, and K layers of those points, each displaced by a distinct vector of norm at most $35 - 28.28 = 6.72$ m, cover it K times with $4K$ discs. So $N = 8$ is not merely sufficient for the region; it is exactly sufficient.

Sensor-set bound, and what the corner certificate does not say. The sensors are not at the corners, so (12) does not transfer to them. It would transfer if the sensor set contained four sensors whose coverage discs were pairwise disjoint, and a sufficient condition for that is one sensor in each of four corner caps of radius below 5.00 m: two sensors in adjacent caps are then at least $80 - 2r$ apart, and their discs are disjoint while $80 - 2r > 2R_{\mathrm{cov}} = 70$ m, the adjacent pair being the binding one (a diagonal pair is $80\sqrt{2} - 2r$ apart). The condition is sufficient and not necessary — a packing of four sensors more than 70 m apart can occur anywhere and not only near the corners — but a random draw essentially never meets it: a cap of that radius is 0.31% of the square, and a uniform draw of 20 sensors lands one in each of the four caps with probability 9.3×10^{-6}. The applicable bound is (11) applied to the realised sensor set. Measured over 30 random draws (`analysis/feasibility.py`), the packing number has mean 2.133 and range 2–3, so the bound is $N \geq 4.27$ on average and $N \geq 6$ in the worst case, and $N = 5$ is *excluded* for 4 of the 30 draws (13%). On the scenes on which the teleporting oracle was measured it reaches $CK = 0.990$ at $N = 5$, so the sensor requirement does not obstruct $N = 5$ there; what that measurement does not do is certify feasibility for the remaining draws, and the correct statement is a probabilistic one, not a certificate. We state this rather than the convenient one: the configuration is *provably* infeasible for the region, and *probably* infeasible for a minority of sensor draws.

The corrected configuration is feasible for both requirements. $N = 8$, $K = 2$, $R_{\mathrm{cov}} = 35$ m satisfies (12) with equality and the construction above, so every sensor is K-covered for *every* sensor layout, not merely for the ones that were drawn. The teleporting oracle of Section III-G confirms it, reaching $CK = 1.000$ with a connected placement.

The area-surplus criterion certifies nothing. The metric $\sigma = N\pi R_{\mathrm{cov}}^2/(K|\Omega|)$ equals 1.503 at $N = 5$ and exceeds one, and the configuration is nonetheless infeasible for the region. The reason is that area is a measure condition and covering is a pointwise one: N discs can be piled over a small patch, satisfying $\sigma > 1$ while most of Ω is uncovered, and conversely a clustered sensor set can be covered with $\sigma < 1$. In particular $\sigma \geq 1$ is *necessary* for K-covering the region and is *not even necessary* for K-covering a sensor set, and it is never sufficient: at $N = 7$, $K = 2$ it is 2.10 and the region bound (12) still excludes the configuration. A feasibility certificate has to be a covering argument.

What the degenerate metric does, exactly. Let $\mathcal{F}$ be the set of configurations in which every sensor is K-covered and the communication graph is connected. The service-continuity metric of Table X satisfies

$$\mathcal{F} = \varnothing \quad \Longrightarrow \quad SC(\pi) = 0 \text{ for every policy } \pi, \tag{13}$$

because $SC(\pi) > 0$ requires the trajectory to visit $\mathcal{F}$ at some instant. The converse does not hold: a poor policy on a feasible instance may also score zero, so observing $SC(\pi) = 0$ does not prove infeasibility. The statement is therefore about the best achievable value $\max_\pi SC(\pi)$ and not about the value any particular policy realises: a poor policy on a perfectly feasible instance also scores $SC = 0$. When $\mathcal{F} = \varnothing$ the metric is uninformative, and the time-averaged coverage deficit of the same table is not, which is the point of reporting it alongside.

Corrected, feasible deployments: $N = 8$, $K = 2$, $R_{\mathrm{cov}} = 35$ m (provably feasible for both requirements, and tight against the region bound) or $N = 5$, $K = 2$, $R_{\mathrm{cov}} = 45$ m, for which four discs at the midpoints of the region's sides and one at its centre cover every point of the region at least

twice, the worst second-nearest distance being $44.72\,\mathrm{m}$ against the $45\,\mathrm{m}$ offered. All trained results in this report use the reported eight-UAV deployment; an earlier comparison of the two deployments was measured with the pre-correction learner and is not claimed.

Achievability ceiling. A teleporting placement oracle (`analysis/feasibility.py`) places the N UAVs anywhere, at no motion cost and ignoring energy, and upper-bounds the coverage fraction C_K of any policy, because the per-step coverage fraction of any trajectory is bounded by the best static placement. It therefore separates "the policy is weak" from "the task is infeasible", and it is the ceiling against which the reported coverage is read. Two limits on the claim are worth stating. The oracle is an upper bound on C_K and *not* on the service-continuity metric, which additionally requires the requirement to hold at every instant of a kinematically constrained trajectory; and it does not certify that a feasible trajectory achieving the certified coverage exists. That last point is checked rather than assumed: the construction of (12) is admissible under the simulator's own safety filter — its communication graph is connected, it has no articulation point, and its eccentricity is 3, within the H-hop rule — so the certified configuration is reachable by a policy and the ceiling is comparable with what a policy can achieve. The normalisation by the oracle is, at this configuration, the identity, since the ceiling is 1.000 for every sensor layout; its information is that the task is coverable at all with $N = 8$. Since the construction is itself admissible under the safety filter, the *constrained* ceiling is the same 1.000: the constraint removes no headroom, so there is no second ceiling to compute, and the normalisation by the oracle is meaningful at this configuration rather than vacuous. At the corrected configuration the value is $C_K = 1.000$ and it is attained for *every* sensor layout, because the construction of (12) covers the whole region, so the ceiling is deterministic rather than a random variable over layouts.

Non-degenerate metrics. The service-continuity metric "fraction of time all sensors are K-covered *and* the graph is connected" is identically zero for every policy when $\mathcal{F}$ is empty, by (13). It is retained for comparison, and reported alongside two quantities that are not. The first is the time-averaged coverage deficit $\frac{1}{T}\sum_t(1-\overline{\mathrm{CK}}_t)$, which is graded: it gives partial credit for a near miss, where the binary flag gives none, and it is the better-behaved quantity as a training signal. We note that it is, exactly, the complement of the coverage fraction C_K, so it is a change of convention rather than an independent measurement, and that it is blind to the connectivity half of the requirement. The second is the maximum refresh time, the longest run of consecutive steps with any under-covered sensor, which is not a function of C_K: a controller can hold a high average coverage while leaving one sensor dark for a long time, and the refresh time is what detects that. It is the maximum refresh time, rather than the deficit, that carries the information the binary flag cannot.

H. Connectivity Model

The inter-UAV communication graph is

$$G_t = (V_t, E_t), \qquad E_t = \{(i,j) : \|\mathbf{p}_i - \mathbf{p}_j\|_2 \le R_{\mathrm{comm}}\}. \tag{14}$$

The ground data sink is not a vertex of the graph: the connectivity the paper measures is the connectivity of the active-UAV graph, and the sink is assumed to stay within R_{comm} of at least one UAV, which is an assumption and not a guarantee. For analysis, the exact articulation-point indicator, where G_t^{-i} denotes G_t with vertex i and its incident edges removed, is

$$\kappa_i(t) = \mathbb{1}\left[G_t^{-i} \text{ is disconnected}\right]. \tag{15}$$

The average criticality is

$$\mathrm{Crit}_t = \frac{1}{|V_t^{\mathrm{act}}|}\sum_{i\in V_t^{\mathrm{act}}} \kappa_i(t). \tag{16}$$

The exact graph quantity is used by the simulator for evaluation. The actor receives the local proxy ρ_i rather than the full graph.

I. The Radio Model, and What It Gates

The graph of (14) and the coverage condition of (7) are both geometric, and both are gated by a radio model: a UAV counts towards a sensor's coverage, and two UAVs count as adjacent, only when the link between them carries a usable service rate. The gate is what makes the discs physical rather than notional, and it is stated here because the two conditions above are otherwise read as purely geometric.

The air-to-ground path follows the Al-Hourani model [31]: for a UAV at altitude z_0 and a slant range d_3 to the ground node, the elevation angle $\theta = \arcsin(z_0/d_3)$ gives the probability of an unobstructed path,

$$P(\mathrm{LoS}) = \frac{1}{1 + a\,e^{-b(\theta - a)}}, \qquad a = 9.61, \quad b = 0.16, \tag{17}$$

and the path loss blends a log-distance law for the clear path with one for the obstructed path,

$$\begin{aligned} PL(d) &= P(\mathrm{LoS})\,PL_{\mathrm{LoS}}(d) + \big(1 - P(\mathrm{LoS})\big)\,PL_{\mathrm{NLoS}}(d), \\ PL_{\cdot}(d) &= PL(d_0) + 10\,n_{\cdot}\log_{10}\frac{d}{d_0}, \end{aligned} \tag{18}$$

with $PL(d_0) = 20\log_{10}\!\big(4\pi d_0 f/c\big)$, $d_0 = 1$ m, $n_{\mathrm{LoS}} = 2.0$ and $n_{\mathrm{NLoS}} = 3.5$ [32]. A shadowing term with a standard deviation of 4 dB is drawn once per episode, so that shadowing is a property of the scene and not of the step, and the inter-UAV links share one draw per pair because a link is one physical link.

The received power is $P_{\mathrm{tx}}\,10^{-PL/10}$, with $P_{\mathrm{tx}} = 0.5$ W at $f = 2.4$ GHz, and the noise floor is the thermal noise of a 20 MHz channel with the receiver's noise figure. The signal-to-noise ratio selects an achievable rate from the IEEE 802.11 OFDM MCS table [33], and a link is *usable* when that rate is at least 6.5 Mbit/s, the lowest rate in the table. Writing

rate_{id} for the rate that the access link from UAV i to sensor d achieves, and rate_{ij} for the rate of the inter-UAV link, the two conditions of this paper are therefore the conjunction of a geometric and a radio condition: the coverage count of (8) is taken over the summands of (19),

$$n_d(t) = \sum_{i \in \mathcal{V}_t^{\mathrm{act}}} \underbrace{\mathbb{1}[\|\mathbf{p}_i - \mathbf{q}_d\|_2 \le R_{\mathrm{cov}}]}_{\text{geometry}} \underbrace{\mathbb{1}[\mathrm{rate}_{id} \ge 6.5]}_{\text{radio}}, \quad (19)$$

and the communication graph of (14) carries an edge when $\|\mathbf{p}_i - \mathbf{p}_j\|_2 \le R_{\mathrm{comm}}$ *and* $\mathrm{rate}_{ij} \ge 6.5$ Mbit/s. The access link is noise-limited, because the UAVs are assigned orthogonal channels; a co-channel term is implemented and is switched off, since K-coverage asks whether K UAVs are *able* to serve a sensor, not whether all K transmit on one resource at the same instant.

Two consequences follow. First, the discs are the geometry and the radio is the budget, and which of the two binds is a measurement rather than an assumption: Section **??** measures the link margin at the two operating radii and finds it wide, so the feasibility certificate of Section III-G and not the radio is what constrains the fleet. Second, because the gate is in the model, the coverage reported here is coverage of *serviceable* links and not of discs.

J. Battery Model

Let $B_i^{\max}$ be the battery capacity in joules and $E_i(t) = b_i(t) B_i^{\max}$ the remaining energy. The slot energy update is

$$E_i(t+1) = \mathrm{clip}(E_i(t) - P_i(t)\Delta t + P_{\mathrm{ch},i}(t)\Delta t,\ 0,\ B_i^{\max}), \quad (20)$$

where the action-dependent consumption is

$$P_i(t) = P_{\mathrm{hover}} \mathbb{1}[a_i = \mathrm{Hover}] + P_{\mathrm{move}} \mathbb{1}[a_i \in \{+x, -x, +y, -y\}] + P_{\mathrm{comm}}. \quad (21)$$

The charge power is zero unless the UAV is physically docked and granted service at a pad. A return threshold is defined by

$$E_i^{\mathrm{req}}(t) = E_i^{\mathrm{pad}}(t) + E_i^{\mathrm{rejoin}}(t) + E^{\mathrm{reserve}}, \quad (22)$$

and a return is mandatory when $E_i(t) \le E_i^{\mathrm{req}}(t)$.

Two properties of this model are easy to state and easy to implement incorrectly, and both were implemented incorrectly in the first version of the simulator, so both are pinned by tests. First, the return is a *physical* transition and not a decision available to the policy: the simulator makes it, so that the reward, the metrics and the charging arbitration can all see it. If the simulator does not make it, then a learned policy has no way to express the return at all, the arbitration is never entered, and the reserve mechanism has nothing to recover — the three mechanisms this paper proposes are then present in the model and absent from the experiment. Second, a UAV that is docked and charging has its rotors idle and draws only the avionics and communications baseline; if it is billed the hover power, the pack loses more energy to the rotors than the pad puts back, no UAV can ever recover, and the charging coordination is unusable. A UAV whose pack empties away from a pad is out of service: it can no longer fly, and it must not continue to count towards coverage or towards the connectivity graph. Figure 4 shows the battery and failure state machine: the transitions between flying, returning to charge, charging and failing, and the point at which the reserve UAV takes over.

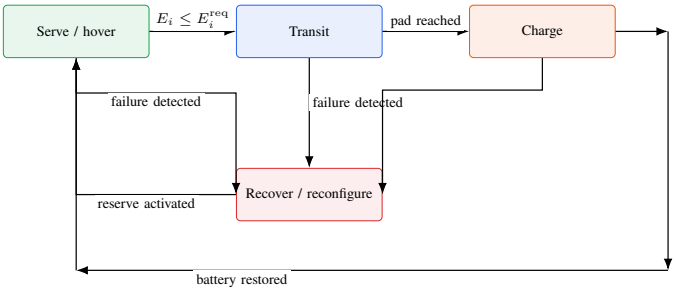


Fig. 4. Battery and failure state machine. Energy urgency triggers a return-to-charge transition; a detected failure transfers the mission to a recovery phase in which the reserve UAV can be activated when feasible.

K. Charging Coordination

Charging is a scarce resource. Let $y_{is} \in \{0, 1\}$ denote whether UAV i is assigned to pad s. For a charge request, define a local assignment cost

$$c_{is} = \alpha \tilde{d}_{is} + \beta \tilde{q}_s + \mu \tilde{u}_i, \quad (23)$$

where $\tilde{d}_{is}$ is normalized distance, $\tilde{q}_s$ is normalized queue length, and $\tilde{u}_i$ is normalized battery urgency. The arbitration layer solves

$$\begin{aligned} \min_{\mathbf{y}} \quad & \sum_i \sum_s c_{is} y_{is} \\ \text{s.t.} \quad & \sum_s y_{is} \le 1,\ \forall i, \\ & \sum_i y_{is} \le 1,\ \forall s, \\ & y_{is} \in \{0, 1\}. \end{aligned} \quad (24)$$

Because only charge requests are handled, this is a small scheduling problem rather than a centralized trajectory controller. A Hungarian-style assignment implementation is sufficient for the expected fleet size.

L. Safety Constraints

The environment enforces

$$\begin{aligned} & \|\mathbf{v}_i(t)\| \le v_{\max}, \qquad \mathbf{p}_i(t) \in \Omega, \\ & \|\mathbf{p}_i(t) - \mathbf{p}_j(t)\| \ge d_{\mathrm{safe}},\ \forall i \ne j. \end{aligned} \quad (25)$$

An action is masked if it violates a locally predictable hard constraint such as workspace bounds, speed, minimum separation, or invalid charging. The energy rule deserves a sentence of its own, because a weak version of it is a mechanism rather than a detail: a UAV that may depart with a flat battery will fly until its pack is empty, and an exhausted UAV is removed from the active set, taking its share of the coverage and of the communication graph with it. The mask therefore requires a

UAV to retain, after the move, the energy to reach a charging pad plus the reserve margin, computed from where the UAV *would be*. The rule is one-sided in the UAV's favour, because a UAV that is already below the threshold must still be allowed to move towards a pad; blocking it there would strand it certainly. Connectivity is handled differently, and the distinction matters. The *policy* is not assumed to predict connectivity from one-hop observations; it receives the local bridge-risk feature ρ_i and the team reward penalises connectivity fragility, so nothing in the learning signal claims a global guarantee from purely local information. The *simulator*, however, is a safety filter: it evaluates the prospective communication graph for every candidate action and masks any move that would leave the swarm unable to remain connected, including the H-step lookahead of Section IV. The guarantee therefore comes from the safe-filter architecture, in which the filter has access to the true state, and not from the policy's own foresight. The distinction is the standard one between learning which action to take and being told which actions are admissible.

M. Failure and Recovery

A failure event changes F_i from 0 to 1 at the detection instant t_f. One reserve UAV is maintained with sufficient energy for deployment when feasible. After a detected failure, the reserve is activated and the active UAVs continue using their decentralized policies. The recovery time is defined as

$$\tau_f = \inf\left\{\tau \geq 0 :\ n_d(t_f + \tau) \geq K,\ \forall d\right\}, \tag{26}$$

when recovery is achieved before the evaluation horizon. If recovery is impossible under the available fleet, the event is reported as an unrecovered failure rather than assigning an artificial recovery time.

A second mode of loss of an active UAV is exhaustion rather than failure: a UAV whose pack empties while it is away from a charging pad can neither fly nor serve, and it is removed from the active set. Such a UAV is out of service rather than failed, and the same reserve mechanism can re-seat it. This distinction is not cosmetic. If an exhausted UAV were left in the active set it would continue to count towards coverage and towards the connectivity graph while being unable to do either, and the reported coverage would be optimistic by exactly the number of sensors such a UAV was covering.

A recovery-time guarantee is therefore conditional: the reserve mechanism can reduce the time required for reconfiguration, but it cannot mathematically guarantee restoration for arbitrary sensor layouts, fleet sizes, or multiple simultaneous failures. This distinction is important for a realistic system model.

N. Primary Objective and Reward Function

The project formulation adopts a "service first, efficiency second" objective. The primary quantity is persistent service loss, represented by the fraction of IoT sensors that are below the required redundancy level. Let

$$J_{\text{out}}(\pi) = \mathbb{E}_\pi\left[\sum_{t=0}^{T-1} \gamma^t U_t\right], \tag{27}$$

TABLE V
TEAM REWARD TERMS AND THEIR INITIAL WEIGHTS.

Term	Definition	Weight
U_t	Under-coverage rate, (10)	$\lambda_O = 1.00$
E_t^{cons}	$\sum_i \Delta E_i(t) \,/\, (NB^{\max})$	$\lambda_E = 0.10$
D_t^{deg}	1 if any service requirement is violated	$\lambda_D = 2.00$
W_t^{mov}	distance flown / distance that changed coverage	$\lambda_W = 0.05$
R_t^{rec}	normalized recovery delay during recovery	$\lambda_R = 0.30$
Crit_t	Articulation-point average, (16)	$\lambda_C = 0.50$
B_t	mean fractional shortfall below the return threshold	$\lambda_B = 2.00$

where U_t is the under-coverage rate in (10). The control policy therefore prioritizes minimizing long-horizon service loss rather than minimizing energy alone.

The corresponding multi-objective optimization is

$$\begin{aligned}\min_{\pi}\quad & J_{\text{out}}(\pi) + \lambda_E J_E(\pi) + \lambda_D J_D(\pi) \\ & + \lambda_W J_W(\pi) + \lambda_R J_R(\pi) + \lambda_C J_C(\pi),\end{aligned} \tag{28}$$

subject to the physical constraints in Section III-L. The secondary terms represent energy expenditure, degraded-service time, unnecessary movement, recovery delay, and topology criticality, respectively.

For reinforcement learning, the constrained objective is converted into a shared team reward:

$$\begin{aligned}r_t = -\big(&\lambda_O U_t + \lambda_E E_t^{\text{cons}} + \lambda_D D_t^{\text{deg}} + \lambda_W W_t^{\text{mov}} \\ &+ \lambda_R R_t^{\text{rec}} + \lambda_C \text{Crit}_t + \lambda_B B_t\big).\end{aligned} \tag{29}$$

Since the learner maximises expected return while the control objective of (28) is a minimisation, the team reward of (29) is the negated weighted cost, and the two are equivalent up to sign. Table V defines each term and gives the initial weights used in this work. The weight λ_O is chosen to dominate the secondary efficiency terms, so that the learned controller does not sacrifice persistent service merely to reduce energy consumption or flight distance. The individual objective components remain separate evaluation metrics rather than being hidden inside a single final score. One consequence is worth stating because it bears on how the results should be read: $\frac{1}{T}\sum_t U_t = 1 - \text{CK}$ exactly, so the coverage term of the reward *is* the headline coverage metric. A learned controller that loses the comparison has therefore lost on the largest term of its own training objective, which locates the shortfall in the part of the objective the reward prices most heavily. It does not by itself attribute the shortfall to the learning pipeline: the objective also prices energy, distance and connectivity, the oracle establishes that the instance is coverable but says nothing about what a decentralised policy class can achieve on it, and a hand-designed rule with access to the simulator's state is not a compute-matched opponent. The consequence for the results is that the components that are not in the reward — the joint service-continuity flag, the refresh time, the energy, the stranding count — carry the diagnostic weight in Section VIII.

All seven quantities are dimensionless and lie in $[0, 1]$, so the weights are directly comparable; U_t and Crit_t are already normalized in (10) and (16). Two of the scalings are worth stating precisely, because they are much weaker than the table's layout suggests. The energy term is normalized by $NB^{\max}$, so its largest possible value per step is 8×15 J divided by 240,000 J, that is 5×10^{-4}, and with $\lambda_E = 0.10$ it contributes at most 5×10^{-5} to the cost against a coverage term of order one: the energy term is numerically inert at this scale, and the ordering $\lambda_D \geq \lambda_O \gg \lambda_E$ is far stronger than a comparison of the weights alone suggests. The threshold term is normalized by $B^{\max}$, whereas the shortfall it measures is bounded by $E^{\mathrm{reserve}} + E_i^{\mathrm{pad}}$, so its largest value is about 0.13; the observation carries the same quantity normalized by E_i^{req}, which is the scale on which it is naturally read, and the two differ by a factor of seven. Finally, W_t is the fraction of the distance flown that did not buy coverage, and it is defined to be zero on a step in which nothing moved, so that holding station while a UAV charges is not counted as waste; the definition is a convention and we state it rather than leave it implicit. The values are initial settings to be tuned on a validation set. The implemented values raise λ_D above λ_O so that a degraded-service step is penalised more heavily than a merely under-covered one; λ_C is likewise raised so that connectivity loss is not drowned out by coverage. The ordering on which the formulation depends is $\lambda_D \geq \lambda_O \geq \lambda_C \geq \lambda_R > \lambda_E \geq \lambda_W$, with the threshold term's weight $\lambda_B = 2.00$ equal to λ_D.

a) The energy-threshold term, and why a consumption term is not enough.: E_t^{cons} prices what is *consumed*. That is not the quantity that determines whether a UAV survives the mission: flying 30 m to a charging pad costs about 480 J, which is 1.6% of a 30 kJ pack, so the consumption term contributes about 0.0016 to the cost, whereas the consequence of not going — a UAV that cannot reach a pad leaves the swarm, stops covering, and may fragment the graph — is paid *late* and by the team. A controller that is only told what it has spent therefore has no reason to turn back before the consequence arrives. B_t is the mean fractional shortfall below the return threshold of (22), i.e. the quantity the observation already carries as the return-urgency feature, and it is the term the paper's own metric table asks for when it lists "minimum battery level, and threshold violations" under Energy. Section VIII reports a controlled measurement of the effect of this term and of the alternative hypothesis that the energy constraint is simply too tight for the episode length. Table VI separates the two kinds of constraint: the hard ones the filter enforces on every candidate action, and the trade-offs the reward asks the policy to make.

TABLE VI
HARD CONSTRAINTS VERSUS LEARNED TRADE-OFFS.

Constraint	Mathematical form	Enforcement / evaluation
Speed	$\|\mathbf{v}_i\| \leq v_{\max}$	Built into the action model and feasibility mask.
Workspace	$\mathbf{p}_i \in \Omega$	Action mask prevents leaving the allowed region.
Separation	$\|\mathbf{p}_i - \mathbf{p}_j\| \geq d_{\mathrm{safe}}$	Action mask prevents predicted collisions.
Battery reachability	$E_i \geq E_i^{\mathrm{pad}} + E^{\mathrm{reserve}}$ after the move	The mask computes the cost of reaching a pad from where the UAV *would be*, and blocks a departure that would leave it unable to get home. The rule is one-sided in the UAV's favour: a UAV already below the threshold may still move towards a pad.
Charging capacity	At most one UAV per pad	Explicit pad assignment / scheduler.
K-coverage	$n_d(t) \geq K$	Collective service objective and evaluation metric; not guaranteed by a single-agent mask.
Connectivity	G_t connected	Topology-aware reward and connectivity metric.
Recovery	$\tau_f \leq \tau_{\max}$ when feasible	Measured scenario requirement, not an unconditional guarantee.

TABLE VII
INITIAL SIMULATION PARAMETERS.

Parameter	Symbol	Value
Active UAVs	N	8 (5 was infeasible for $K = 2$)
Reserve UAVs	N_R	1
IoT sensors	M	20
Operating region	Ω	80×80 m^2
Coverage radius	R_{cov}	35 m
Communication radius	R_{comm}	40 m
Nominal altitude	z_0	50 m
Coverage target	K	2
Control interval	Δt	0.2 s
Maximum speed	$v_{\max}$	5 m/s
Safety distance	d_{safe}	5 m
Battery capacity	$B^{\max}$	30,000 J
Hover power	P_{hover}	50 W
Move power	P_{move}	75 W
Avionics and comms power	P_{elec}	5 W
RF transmit power	P_{tx}	0.5 W
Charge increment	–	0.0002 of the pack per step
Episode length	T	2000 steps (400 s)
Initial deployment	–	connected cluster in a corner

O. Parameter Settings and Scope

Table VII gives the initial simulation configuration derived from the project layout. These are implementation parameters for the proposed simulator rather than experimentally calibrated real-flight values.

The control interval is chosen so that the relative displacement between two consecutive steps, $2v_{\max}\Delta t = 2$ m, is smaller than the safety distance $d_{\mathrm{safe}} = 5$ m; a coarser step would allow a closing pair to cross the separation constraint between two samples. The charge increment is quoted per step and is therefore scaled with Δt. The resulting charge time is shorter than a real constant-current pack of this capacity would require, because a real pack would not complete a full charge within a tractable episode; this is stated as a modelling assumption rather than a physical prediction, and it means the cost of charging contention in the energy results of Section **??** is a lower bound. The nominal altitude is consistent with the coverage radius: a 74° downward sensor at 50 m yields a footprint half-width of 37 m. The two power entries are distinct: P_{elec} is the total avionics and communications budget, whereas P_{tx} is the radiated transmit power, which is bounded by the 1 W (30 dBm) regulatory ceiling and is two orders of magnitude below the avionics budget. The chosen $v_{\max} = 5$ m/s is conservative but not arbitrary: Luna et al. [20] report a real three-UAV coverage mission flown at 5 m/s cruise

and 20 m altitude, and Datsko et al. [19] measure a range-optimal cruise speed of 8.39 m/s for the airframe they study.

a) The initial deployment and the episode length are experimental choices, and they are stated as such.: Two parameters of Table VII deserve an explicit justification, because the results are sensitive to both and a reader should be able to see why they were chosen.

Initial deployment. The swarm starts as a connected cluster in one corner of the region, not as a deployed formation. The reason is measurable: a centred ring deployment already covers 17 of the 20 sensors at $t = 0$, so every controller — including one that never moves — scores $CK = 0.850$, the achievable range is roughly $[0.85, 1.0]$, and the comparison cannot separate the methods. The clustered deployment is connected at $t = 0$, so the connectivity invariant remains checkable from the first step, but it covers only 3 of the 20 sensors, so the controller has to deploy across the field while holding $K = 2$ redundancy. This is the standard arrangement for a persistent-coverage benchmark, and the pre-deployed arrangement is retained as scenario S9 so that the sensitivity is reported rather than hidden.

Episode length. The episode is 2000 control steps, i.e. 400 s of flight. At the 400-step episode used in an earlier revision — 80 s of flight, which is 13% of the pack's 600 s endurance — the pack never depleted: the return-to-charge transition never fired, the charging arbitration was never entered, the reserve UAV had nothing to recover, and the three mechanisms the paper proposes were never exercised. A sweep over the candidate configurations (`analysis/energy_regime.py`) shows that at 2000 steps the energy constraint binds — 213 return-to-charge transitions and 0.4 stranded UAVs per episode under the Reactive heuristic — while the mission remains achievable. The report's 10 000-step target is a different question, and the results here are explicitly for the 400-second episode.

Within scope are spatial K-coverage, range-limited UAV connectivity, battery depletion, charging contention, single-UAV failure recovery, collision avoidance, and decentralized motion-policy execution. The primary model intentionally abstracts small-scale multipath fading, detailed aerodynamic dynamics, obstacles, wind, battery aging, GPS denial, sensor failures, and multiple simultaneous UAV failures. These can be introduced as subsequent stress-test layers after the base control problem is validated.

b) Consistency of the physical parameters.: The audit of Section **??** also records the derived physical quantities, so that the modelling choices are explicit rather than implicit. The endurance implied by $B^{\max}$ and P_{hover} is $30{,}000/50 = 600$ s (10 min); the power ratio $P_{\text{move}}/P_{\text{hover}} = 1.5$ is conservative against the 1.09 measured by Datsko et al. [19] for energy-optimal multirotor flight; and the charge increment of 2×10^{-4} of the pack per step corresponds to a full charge in 1000 s, i.e. a 3.6 C rate, which is above the 1 C typical safe charge rate of hobby LiPo and is therefore an explicit modelling assumption rather than a physical prediction. Two further observations matter for the interpretation of the results. First, the separation constraint is enforced at the sample rate: two UAVs closing head-on at $2v_{\max}$ have a braking distance of $v_{\text{close}}^2/(2a) = 25$ m at $a = 2$ m/s^2, five times d_{safe}, so the mask guarantees sample-time separation and not braking. Second, the energy budget of the reported episode is tight rather than generous. A pack of 30,000 J at the moving draw of 80.1 W lasts 1873 steps of continuous flight and nothing else, so the fleet can sustain the 2000-step episode only at a duty cycle of about 89% against the E^{reserve} margin, and the charging rate of 0.0002 of the pack per step is a 3.6C charge, which is an explicit modelling assumption of this work and not a physical prediction; the consequence is that the quantity the charging machinery has to arbitrate is genuinely scarce, and a controller that does not manage it will strand its UAVs. A third observation concerns the connectivity constraint: at $N = 8$ on an 80×80 m field the inter-UAV graph is not always dense — measured over a 2000-step flight in the deployed regime, the mean node degree is 3.63 of a possible 7, the graph is complete in only 12% of steps, and nodes of degree ≤ 1 — leaves of the graph, which are not articulation points but mark how thin the swarm's connectivity is — occur in 23% of UAV-steps. Connectivity is therefore a live constraint throughout the mission and not only after a failure, which is what makes the H-step mask and the reserve mechanism load-bearing rather than decorative. All four quantities are reported rather than silently adjusted, because changing them would change the measured results and the results are the claim.

IV. Proposed ML Algorithm

A. CTDE MAPPO

The proposed algorithm uses parameter-shared MAPPO. During training, the centralized critic observes $\mathbf{s}_t$ while every actor uses $\mathbf{o}_i(t)$. Figure 5 shows the loop: the environment and the control loop, the training-only feedback to the critic, and the transfer of the learned actor to decentralized execution. During deployment, the critic is removed and each UAV evaluates the same shared policy:

$$a_i(t) \sim \pi_\theta\left(a_i \mid \mathbf{o}_i(t)\right). \tag{30}$$

Parameter sharing is appropriate when (i) the environment is equivariant to agent permutations—identical observation and action spaces and identical transition/reward structure—and (ii) each agent's own observation *breaks the permutation symmetry*, so that a shared network can still specialise (Foerster et al. [21]). Condition (ii) does not require a literal index: relative coordinates to the other agents break the symmetry just as well, because they distinguish the agents. What it does require is that no two agents in the same physical situation receive the same observation, and the consequence is a statement about the hypothesis class rather than about optimisation: if $\mathbf{o}_i = \mathbf{o}_j$ then a shared policy is a function and $\pi_\theta(\cdot|\mathbf{o}_i) = \pi_\theta(\cdot|\mathbf{o}_j)$, so no amount of training can make the two agents differ. The first version of $\mathbf{o}_i$ did not satisfy condition (ii) — it distinguished the agents only through their individual position, battery and mode, which in a symmetric deployment are nearly identical across the swarm — and the measured consequence was that the deterministic policy chose nearly the same action

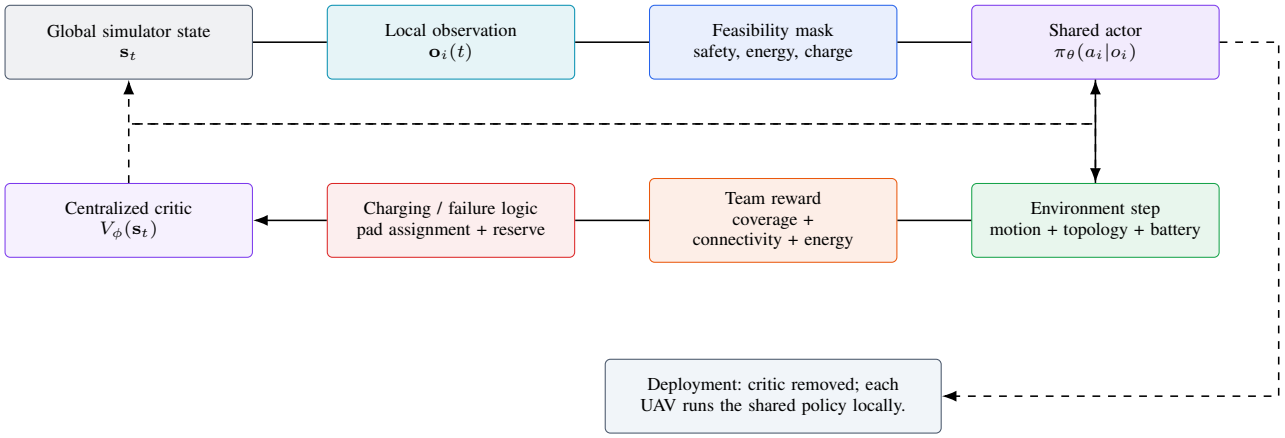


Fig. 5. MAPPO training and deployment pipeline. Solid arrows represent the environment/control loop. Dashed arrows denote training-only critic feedback and transfer of the learned actor to decentralized execution.

for every UAV and the swarm moved as a rigid body, so the shared-parameter claim could not be sustained on the evidence. We therefore treat $\mathbf{1}_i$ as part of the model rather than as an implementation detail, and Section VIII reports the controlled comparison. The known limitation (Fu et al. [28], Theorem 4.2) is that a shared policy provably cannot learn an optimal policy for non-symmetric tasks such as the XOR game; this does not apply here because the swarm is homogeneous. If grounded or docked UAVs with heterogeneous roles are introduced, parameters must not be shared.

For a collected rollout, the temporal-difference residual is

$$\delta_t = R_t + \gamma V_\phi(\mathbf{s}_{t+1}) - V_\phi(\mathbf{s}_t), \tag{31}$$

and the generalized advantage estimate (GAE) is

$$\widehat{A}_t = \sum_{l=0}^{T-t-1} (\gamma\lambda)^l \delta_{t+l}. \tag{32}$$

A fixed-length rollout ends either in a genuine terminal state or in a time-limit truncation, and the two must not be conflated. At a truncation boundary the last state is not absorbing, so the advantage recursion must cut without treating the state as terminal; the correct treatment is to bootstrap the temporal-difference residual from $V(\mathbf{s}_T)$ rather than from zero [34]. The size of the error is explicit: replacing the bootstrap by zero changes the value target by $-\frac{\gamma(\gamma\lambda)^{T-t}}{1-\gamma\lambda} V(\mathbf{s}_{T+1})$, so the target is biased downward whenever the continuation is worth more than zero and upward when it is worth less, and the sign follows the sign of the continuation value. The claim is about the value target and not about a fixed direction of policy improvement, since a mis-estimated advantage near zero can push an update either way. The bootstrap is unbiased for V^π only in the idealisation that the critic is exact and the truncation is exogenous; with an approximate critic it inherits the critic's error at the boundary, scaled by γ^{T-t}. Conflating the two nevertheless injects a systematic error, and it did so in this work: the simulator initially reported the end of the mission time limit through the same flag it used for a genuine dead end, so every episode boundary was bootstrapped from zero. The simulator now reports termination and truncation separately — the mission time limit is a truncation, and only the loss of every UAV is a terminal state — and the rollout consults that flag rather than inferring termination from the episode-end flag. The distinction is pinned by unit tests, one of which asserts that a time-limit episode is reported as a truncation and not as a termination, and another that asserts the rollout reads the environment's flag rather than re-deriving it. The probability ratio is

$$r_t(\theta) = \frac{\pi_\theta(a_t \mid o_t)}{\pi_{\theta_{\text{old}}}(a_t \mid o_t)}. \tag{33}$$

The clipped PPO objective is

$$L^{\text{actor}}(\theta) = -\mathbb{E}_t\big[\min(r_t\widehat{A}_t, \operatorname{clip}(r_t, 1-\epsilon, 1+\epsilon)\widehat{A}_t)\big]. \tag{34}$$

The critic is trained with the clipped MAPPO form (Yu et al. 2022, Appendix A):

$$L^{\text{critic}}(\phi) = \mathbb{E}_t\Big[\max\big((V_\phi(\mathbf{s}_t) - \widehat{R}_t)^2, \big(\operatorname{clip}(V_\phi(\mathbf{s}_t), V_{\phi_{\text{old}}}(\mathbf{s}_t) - \epsilon, V_{\phi_{\text{old}}}(\mathbf{s}_t) + \epsilon) - \widehat{R}_t\big)^2\big)\Big]. \tag{35}$$

where $\widehat{R}_t = \widehat{A}_t + V_{\phi_{\text{old}}}(\mathbf{s}_t)$ is the TD(λ) lambda-return, computed once per rollout and treated as a constant (gradients are not propagated through it). The report matches the reference implementation (`r_mappo.py`), which computes gae + value_preds rather than the Monte-Carlo return-to-go described in MAPPO's Appendix A prose.

a) Value normalisation, and the scale trap inside it.: Value normalisation is one of the components the MAPPA/MAPPO literature identifies as load-bearing, and it was missing from the implementation while the earlier revision of this report asserted it. It is now present, and the details matter. The critic is trained to predict the returns in a *normalised* space defined by running statistics of the return distribution, maintained as an exponential moving average with $\beta = 0.99$:

$$\widehat{R}_t^{\text{norm}} = \frac{\widehat{R}_t - \mu}{\sigma + \epsilon}, \tag{36}$$

where μ and σ are updated from the current rollout's returns. The consequence is easy to get wrong: the critic's *raw* output is then the normalised value, so the temporal-difference residual of Eq. (31), the advantage recursion of Eq. (32), and the bootstrap at a truncation boundary must all *un-normalise* it before use, as $V(\mathbf{s}) = \sigma\,\hat{V}_\phi(\mathbf{s}) + \mu$. Reading the raw critic output at any of those three points is a silent scale error in the advantage estimates: the training loss still decreases, the entropy still falls, and the policy is simply trained against the wrong signal. The implementation therefore exposes a single accessor for the value and the test suite asserts that the rollout uses it rather than the raw output; a second test asserts that the value-clipping auxiliary term compares two quantities in the same space, since the recorded old value is in the original scale.

The advantage is normalised as well, by the mean and standard deviation of the batch. We also implemented a per-agent variant, in which each agent's advantages are normalised over its own history rather than over the pooled batch, and we report the measurement in Section VIII: with a shared team reward every agent's advantage sequence is nearly identical, so the per-agent normalisation is close to a rescaling and does not change the credit assignment, which is the limitation of shared team rewards rather than a normalisation choice.

b) The credit assignment is a property of the reward, not of the normalisation.: With a team reward and a parameter-shared actor, the policy gradient is $\sum_t \sum_i \gamma^t \nabla_\theta \log \pi_\theta(a_t^i|o_t^i)\big(G_t - b(\mathbf{s}_t)\big)$: the factor that weights every agent's score is the *same* number for all of them, so the gradient cannot distinguish an agent whose action helped from one whose action was irrelevant. Giving each agent its own value function does not repair that: it re-centres and re-scales the shared team signal per agent, so the advantage is no longer a single shared number, but it cannot decompose it, because the advantage's dependence on the other agents enters only through the aggregate reward and through the observation, which are symmetric in the other agents. Redistribution of credit requires a per-agent reward, a joint-action critic, or a counterfactual baseline. This is the credit-assignment objection that the per-agent estimators make explicitly: COMA [22] observes that a global temporal-difference error "does not explicitly reason about how that particular agent's actions contribute to that global reward", and GPAE [24] states the objection in MAPPO's own terms, that the shared team advantage "ignores the agent-specific impact on the joint outcome". We state the limitation rather than the conclusion, because the limitation is not a failure: a shared team advantage is a strong baseline on the standard cooperative benchmarks [3], [27]. What the literature does not contain is an evaluation of the question on a persistent-coverage task, and Section VIII-D reports one: the same learner with the advantage made per-agent, and nothing else changed.

B. Action Feasibility Mask

Before the sampled action is executed, the simulator evaluates the predicted next position and applies the following checks: workspace bounds, maximum speed, minimum UAV separation, charge validity, and connectivity-preservation constraints. If a candidate movement would disconnect the communication graph, it is masked in normal operation. The H-step rule is a statement about the mover's *eccentricity*, not about the graph's diameter and not about robustness: it prevents a UAV from drifting to the end of a long chain, since a node at the end of a chain of length greater than H has an eccentricity greater than H and may not legally act from there, and it does not by itself prevent articulation points, which the separate articulation test forbids — a path of seven nodes with the mover at its centre satisfies the rule and has six articulation points. The rule also constrains only the node that is acting, so another agent may extend a chain and raise the first agent's eccentricity without violating that agent's own check, and in a dispersed graph no legal move may exist for some node at all. A rule expressed as a hop radius therefore restricts the topology more than it protects it, as the ring example above shows. During an active recovery event, the connectivity rule remains a hard constraint unless the simulator enters a predefined recovery mode in which temporary reconnection is required by the recovery objective.

This separation between policy learning and hard feasibility is important. The neural network learns how to trade off several feasible actions, while the safety layer prevents the network from selecting actions that violate non-negotiable physical constraints.

The masking is applied at the logit level, before the softmax, so that the policy that is sampled from, the importance ratio that is computed against, and the entropy bonus that is regularised are all the same masked distribution.

a) The filter does not leak, but it does restrict.: Two properties of the filter are measured rather than assumed. First, the graph is connected at every step in which the active set is unchanged: over 2000 steps driven by the learned controller, the graph was disconnected on no step in which the active set was unchanged (`analysis/mask_admissibility.py`). A connectivity availability below 1.000 in the results therefore coincides with UAVs *leaving* the active set — a failure, or a pack that empties away from a charging pad — and not with the filter failing, because a removed UAV takes its share of the coverage and of the graph with it. Second, the filter excludes configurations that a covering argument would admit: the 8-node ring is connected, has no articulation point, and is a valid covering family, but its eccentricity is 4, so the $H = 3$ rule rejects every move that would preserve it. The safety filter thus shrinks the reachable configuration set, and a controller that needs to move into a rejected family is stuck; we report this as a property of the filter and not as a property of the task. This is the mechanism that Huang and Ontañón [30] show to correspond to a valid policy-gradient update rather than a biased surrogate: masking is implemented as a state-dependent differentiable transformation of the action-probability distribution, which is exactly what setting the masked logits to $-\infty$ before the softmax does. We note the limit of that result, which was

Algorithm 1 MAPPO training algorithm

Initialize shared actor π_θ and centralized critic V_ϕ.
for each training episode **do**
  Reset UAV positions, batteries, sensors, charging pads, and failure schedule.
  for $t = 0, \ldots, T-1$ **do**
    Construct global state $\mathbf{s}_t$ and each local observation $\mathbf{o}_i(t)$.
    for each active UAV i **do**
      Construct the feasible-action mask from Section III.
      Sample $a_i(t) \sim \pi_\theta(\cdot \mid \mathbf{o}_i(t))$ over feasible actions.
    end for
    Apply UAV actions and update positions.
    Recompute coverage and communication graph.
    Update battery states using (20).
    if one or more UAVs request charging **then**
      Solve the charging assignment in (24).
    end if
    if a UAV failure is detected **then**
      Activate the reserve UAV if feasible and update active-agent states.
    end if
    Compute R_t from (29); store rollout transition data.
  end for
  Compute GAE advantages from (32).
  Update θ with the clipped PPO objective.
  Update ϕ with the critic loss.
end for
return Shared decentralized actor π_θ.

stated explicitly in the same work and is repeated in the UAV action-masking literature [36]: the validity argument is formulated for a single policy, and extending it to a centralized critic with heterogeneous per-agent masks requires the critic's training distribution to be consistent with the induced joint-action support. We therefore apply the identical mask to the rollout distribution, the stored log-probabilities and the update, and we verify this by construction in the test suite rather than by assumption.

C. Failure-Aware Execution

When a failure event is detected, the environment removes the failed UAV from the active graph, activates the reserve UAV if the reserve is available, and updates the local observations. The reserve UAV uses the same shared actor after insertion into the mission. The other UAVs also re-evaluate their local bridge-risk and coverage information. No prediction of the failure event is required; the controller reacts to the observed state change.

D. Mapping from Model to Algorithm

The algorithm is deliberately aligned with the system model. Sensor positions produce the coverage quantities in (7)–(10); neighboring UAV positions produce the communication graph in (14); battery and action information produce the energy state in (20); charging requests enter the assignment problem in (24); and failure indicators activate the recovery process in (26). Thus, every actor input corresponds to an explicitly defined state quantity and every learned action has a defined environmental effect.

E. Training and Generalization Protocol

Every measurement in this report is produced by one configuration, built by one function, so that the numbers in a table, the numbers in a sentence and the numbers in the claim audit describe the same experiment: $N = 8$ UAVs, $M = 20$ sensors, $K = 2$, $R_{\mathrm{cov}} = 35\,\mathrm{m}$ on an $80 \times 80\,\mathrm{m}$ field, a clustered initial deployment that is connected at $t = 0$, 2000-step episodes, and the parameter set of Table VII.

Training. A run trains for 200 episodes, and each episode is a fresh scene: the sensor layout, the charging-pad placement and the initial positions are drawn from the environment's own stream at the start of every reset, so the policy sees 200 different layouts in one run and not one layout 200 times. The update is PPO with a shared actor and a centralised critic, discount $\gamma = 0.99$, GAE $\lambda = 0.95$, clipping $\epsilon = 0.20$, learning rate 3×10^{-4}, entropy coefficient 0.01, 10 epochs over 4 minibatches per rollout with an early stop on the approximate KL divergence. A run is one training seed; the spread the report quotes is the spread over seeds.

Evaluation, and the error this subsection exists to prevent. The coverage number a training loop prints is not a result. It is measured while the policy is still changing, on the scene of that episode, by an exploratory policy; a rule baseline is fixed, and measuring it on that one scene compares a policy in motion with a policy at rest on a sample of one. The protocol used here freezes the policy first and then rolls it out, and does the same to the rules:

1) the final actor of a run is saved to a checkpoint, and nothing about it changes after that point;
2) the evaluation scenes are the seeds `1000` and upward, which no training run touches, so no evaluation scene was ever trained on;
3) every controller — learned or hand-written — is rolled out on the *same* scenes, so the scene-to-scene variation is common to all of them and cancels in a difference;
4) the headline number of the learned controller is the sampled policy of Eq. (30), which is what is deployed; the argmax policy is recorded alongside it and is not used for a headline.

What this costs is the ability to say anything from a single run: one policy gives one number per scene and 10 numbers overall, and the uncertainty of the mean is read from the number of training seeds and not from the number of scenes. What it buys is that the comparison is between two controllers on the same task rather than between two protocols.

F. Training Hyperparameters

The initial training configuration follows the values in the project presentation. These values are starting points for the

TABLE VIII
INITIAL MAPPO TRAINING CONFIGURATION.

Parameter	Symbol	Initial value
Active UAVs	N	8
IoT sensors	M	20
Discount factor	γ	0.99
PPO clipping	ϵ	0.20
GAE parameter	λ	0.95
Control interval	Δt	0.2 s

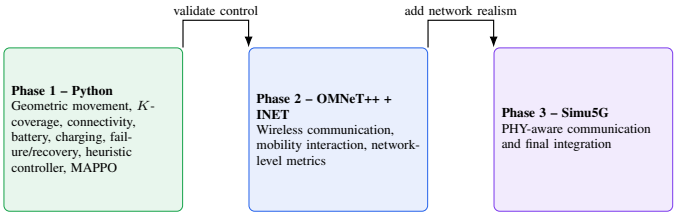


Fig. 6. Staged implementation plan. Phase 1 validates the control loop; Phase 2 adds wireless-network behaviour through OMNeT++/INET; Phase 3 introduces PHY-aware communication through Simu5G.

implementation and can be tuned on a validation set without changing the system formulation. Table VIII gives them.

G. Implementation Plan

The implementation follows the three-stage validation strategy presented in the project plan. The objective is to validate the swarm-control logic before introducing detailed wireless-channel complexity. Figure 6 shows the stages and what each one adds.

Phase 1 contains geometric UAV motion, persistent K-coverage, connectivity, battery dynamics, charging coordination, failure/recovery, a heuristic controller, and MAPPO. Once these components are stable, Phase 2 introduces wireless communication and network-level measurements. Phase 3 then adds PHY-aware communication. The same actor observation/action interface is retained across phases so that increasing simulator fidelity does not require redesigning the learning problem.

a) Execution status.: Phases 1 and 2 have been executed; Phase 3 is scaffolded but not yet run with the learned policy. Phase 1 is implemented in `ml_impl/` with a test suite of 134 cases that pins every reported equation, and the simulator has been exercised over 8 training seeds. Phase 2 is implemented with INET 4.6 and OMNeT++ 6.4 on WSL2 with a Simu5G 1.4.3 stack; the `UAVIoT` network configuration runs to completion and produces scalar and vector outputs, and a dedicated 802.11 physical-layer scenario was built for the validation of Section **??**. The three-stage structure is retained not as a description of a plan but as an audit trail: it records which level of fidelity each reported number was produced at, so that a reader can tell an analytic result from a simulated one and a simulated one from a measured one.

Two things are worth recording about what Phase 1 actually cost, because they bear on how the Phase-3 results should be read. First, the phase is not complete in the sense of the plan: the first implementation of the Phase-1 simulator satisfied every equation of the report and every test, and the coverage numbers it produced were meaningless, because the motion step never applied a move. The defect was found by a diagnostic — the swarm's spread never changed — and not by the equations, the tests or the loss curve. Second, the physical-layer scenario of Phase 2 produced frame counts that did not respond to the transmit power, and the negative control is what exposed it; the instrument was then rebuilt around a statistic that does respond, and the rebuild showed that this statistic is still not a usable instrument for the claim it was wanted for (Section **??**). Both episodes are reported here rather than in a footnote, because a reader deciding whether to trust a result from this pipeline should know that the pipeline's own diagnostics, and not its unit tests, are what caught the two most consequential errors in it.

V. COMPARISON MODELS

The comparison design preserves the practical baselines from the project presentation and adds recent learning-based methods required for a meaningful research comparison. The practical baselines test whether any adaptive coordination is useful; the MARL baselines test whether the proposed CTDE MAPPO design provides advantages over other learning formulations.

The fixed, greedy, and reactive controllers correspond directly to the practical alternatives already identified in the project plan. The two learned baselines are implementations of the method *families* the comparison table names, not reproductions of the cited papers: an independent value-based learner and a masked PPO with a local critic. Both run in the same simulator, on the same observation, with the same action mask and the same team reward as the proposed method, so that the only differences are the learning machinery; where a literature method would require a different input representation or native action space, the adaptation is documented rather than presented as an exact reproduction. Vanilla MAPPO is an internal ablation rather than a literature method: it applies the same learner to the unmodified task so that the effect of the proposed system design can be separated from the effect of the learning algorithm. POGS-QMIX is cited as closely related work and is *not* re-implemented, and the table says so.

All models use the same sensor layouts, initial UAV positions, battery conditions, charging pads, failure traces, episode durations, and physical safety constraints. Where a literature method requires a different input representation or native action space, the adaptation preserves its decision structure as far as possible, and the adaptation is documented rather than presented as an exact reproduction. Results are reported on the common metrics of Table X.

Ablation experiments should additionally remove one proposed mechanism at a time: connectivity-criticality input/reward, reserve-UAV recovery, charging arbitration, and action masking. This distinguishes improvements caused by the MAPPO learning method from improvements caused by the proposed system design.

TABLE IX

COMPARISON MODELS AND RATIONALE. THE "EVALUATED" COLUMN STATES PLAINLY WHICH ROWS WERE RUN IN THE COMMON SIMULATOR; A CITED METHOD THAT WAS NOT RUN IS NOT PRESENTED AS THOUGH IT WERE.

Model	Type	Evaluated	Rationale
Fixed formation + threshold recharge	Rule-based	yes	UAVs retain their initial formation and return to charge using a fixed battery threshold. Establishes a simple reference.
Greedy lowest-battery recharge	Rule-based	yes	The most energy-critical UAV receives priority for charging. Tests energy-first scheduling without learned coordination.
Reactive reconfiguration	Heuristic	yes	Remaining UAVs move toward uncovered sensor regions using deterministic coverage/connectivity rules. Tests fault recovery without MARL.
Independent DQN (value-based family, cf. MAD-DDQN [4])	MARL	yes	One Q-network per agent, each trained on the team reward through its own experience, ϵ-greedy exploration, no centralised critic and no parameter sharing. The value-based reference point.
Masked PPO with a local critic (PPO family, cf. MAHPPO-AM [11])	MARL	yes	Keeps the action mask but drops parameter sharing and the centralised critic: per-agent actor and per-agent local critic. Isolates the contribution of the CTDE design.
POGS-QMIX [14]	MARL	no	A recent persistent-coverage approach incorporating outage history, finite energy, and return-to-charge decisions. Cited as related work; not re-implemented here.
Vanilla MAPPO [3]	MARL	yes	Ablation reference: the same CTDE MAPPO learner with the proposed observation terms, reward shaping, charging arbitration, reserve-UAV mechanism and action mask removed. Isolates the contribution of the system design from that of the learning method.

TABLE X

PERFORMANCE METRICS AND OBJECTIVES.

Metric	Definition and purpose
Persistent K-coverage	$C_K = \frac{1}{TM}\sum_t \sum_d \mathbb{1}[n_d(t) \geq K]$. Measures the fraction of sensor-slot pairs meeting the required redundancy. Higher is better.
Under-coverage	$\bar{U} = \frac{1}{T}\sum_t U_t$. Measures the fraction of sensors below K coverage. Lower is better.
Connectivity availability	$A_G = \frac{1}{T}\sum_t \mathbb{1}[G_t \text{ connected}]$. Measures time for which the swarm is connected. Higher is better.
Connectivity fragility	Average Crit_t and number of partition events. Measures exposure to articulation-point failures and topology breaks. Lower is better.
Service continuity	Fraction of mission time for which both the K-coverage and connectivity requirements are satisfied. Measures end-to-end persistent service. *Note: identically zero on infeasible configurations; supplemented by the non-degenerate metrics below.*
Time-averaged coverage deficit	$\frac{1}{T}\sum_t (1 - \overline{\mathrm{CK}}_t)$. Graded: it gives partial credit for a near miss, where the binary flag gives none. *Exactly* $1 - \mathrm{CK}$, so it is a change of convention rather than an independent measurement, and it is blind to connectivity.
Maximum refresh time	Longest run of consecutive steps in which any sensor has fewer than K covering UAVs. Measures worst-case service gap, and is *not* a function of CK: a controller can hold a high average while leaving one sensor dark for a long time. Lower is better.
Outage severity	Time-integrated under-coverage during degraded and recovery intervals. Measures how much service is lost, not only whether a recovery eventually occurs.
Reserve utilization	Number of reserve activations, activation delay, and reserve mission time. Measures whether replacement capacity is used effectively.
Recovery	Mean/95th-percentile τ_f, recovery success rate, and unrecovered-failure count. Measures fault tolerance directly.
Energy	Total energy consumption, energy per successfully covered sensor-slot, minimum battery level, and threshold violations. Measures efficiency and safety margin.
Charging	Mean wait time, denied requests, number of concurrent requests, and pad utilization. Captures contention and scheduling quality.
Movement	Total distance flown and unnecessary-movement ratio W_t. Quantifies motion overhead.
Safety	Collision events, out-of-bounds events, and hard-constraint violations. These should ideally be zero.
Scalability	Policy inference time and metric degradation as N and M increase. Tests the reason for adopting decentralized execution.
Stability	Mean and standard deviation across independent random seeds; training convergence time/episodes. Measures reproducibility.

VI. PERFORMANCE METRICS

The evaluation should report separate metrics for service quality, connectivity, energy, charging, recovery, movement, safety, and scalability.

a) Which of these the paper reports.: Of the metrics above, this paper reports the persistent K-coverage, the connectivity availability and fragility, the service continuity, the time-averaged coverage deficit, the maximum refresh time, the energy, charging and movement quantities, the reserve activations and the stranding count, the plasticity of the shared actor, and the degradation of the coverage with N and M through the scenario matrix. Three are not reported: the outage severity during a recovery interval, the distribution of the recovery time, and the inference-time scaling. The collision and out-of-bounds counts are prevented by the safety filter rather than measured by it, and the separation constraint can be violated by up to one step of motion when the joint projection reverts a move, so we do not present a zero in a column we did not instrument as a measurement.

A. Evaluation Scenarios

Nine scenarios vary the dominant environmental factors independently before they are combined: sensor density, fleet size, battery capacity, the number of charging pads, R_{comm}, R_{cov}, the time of a single-UAV failure, a combined-stress configuration in which energy scarcity, pad contention, weak connectivity and a failure occur together, and the initial deployment. Each scenario has two variants, and every variant is run.

a) The evaluation protocol, stated exactly.: Every controller is evaluated on the same scenes with the same initial conditions, and the sampling of a deployed policy — Eq. (30) specifies $a_i(t) \sim \pi_\theta(a_i \mid o_i(t))$, so the controller *samples* — is seeded from the evaluation seed alone. Three consequences follow, and they are properties of the protocol rather than of the controllers: an evaluation is a function of (policy, scene); every controller sees the same sampling stream on the same scene, so a difference between two controllers is a difference between policies and not between the luck of the draw; and every number in this paper can be recomputed from the stored policies.

A learned controller is trained several times and reported as the mean over those training runs, with the seed-to-seed spread of the coverage. The rule baselines are deterministic: they have no seed-to-seed spread, and their spread over the evaluation

scenes is reported separately. The two sources of variation are reported separately in Section VIII, because they behave differently: the scene-to-scene variation is common to every controller and cancels in every difference, while the training-seed variation does not.

The single-component arms are *paired*: every variant is trained on the same training seeds as the full configuration, at the same training budget, and differs from it in exactly one component. A difference is therefore measured on the same training runs, and it is reported as a confidence interval on that paired difference rather than as a comparison of two numbers against a threshold. We report the resolution of each arm — the smallest effect it could have detected — because a null result from an arm of a given width is a statement about that resolution and not about the component. The principal conclusions are based on the task-specific metrics in Table X, not on the raw reward curves.

The ablation experiments remove, one at a time: the connectivity-criticality reward term, the degraded-service reward term, the energy-cost term, the under-coverage term, the reserve-UAV mechanism, the charging arbitration, the action mask, and each of the three observation terms. This identifies whether each proposed component contributes an effect that this protocol can resolve. In particular, the comparison between the full proposed framework and vanilla MAPPO is necessary to distinguish improvements caused by MAPPO itself from improvements caused by the domain-specific system design.

Three candidate explanations for a shortfall against the strongest controller in the comparison — the unprivileged reactive rule — are named here, before the results, because the arms of the next section test them one at a time: the content of the local observation, the energy model, and the size of the training budget. Each is tested by a single-factor intervention that changes that one thing and holds the rest of the configuration fixed, and each is reported as a confidence interval on the paired difference, with the resolution at which the arm was measured. A fourth explanation, the credit assignment of a shared team reward, is tested by the cheapest available modification of the learner: a per-agent actor with a per-agent local critic, with the same simulator, the same observation, the same mask, the same team reward and the same budget. We do not claim that these four exhaust the possibilities, and the explanation that survives them was not pre-specified; Section VIII states what the arms do and do not exclude.

VII. Experimental Design Matrix

The evaluation varies the dominant environmental factors independently before combining them. Table XI maps each experiment to a specific research question; every row is run, and the results are reported in Section VIII.

TABLE XI
Experimental scenarios and research questions.

ID	Variable	Research question
S1	Sensor density	How does persistent coverage change as sensing demand becomes sparse or clustered?
S2	Fleet size N	Does decentralized execution retain performance as the number of UAVs grows?
S3	Battery capacity	How sensitive are coverage and recovery to energy scarcity?
S4	Charging pads S	How does pad contention affect waiting time and mission continuity?
S5	R_{comm}	How robust is the policy when connectivity becomes marginal?
S6	R_{cov}	How does the controller respond to tighter coverage geometry?
S7	Failure time/location	How quickly can the swarm recover from early/late and central/peripheral failures?
S8	Combined stress	Can the policy jointly handle energy scarcity, weak connectivity, pad contention, and a failure?
S9	Initial deployment	How much of the achieved coverage is a property of the deployment rather than of the controller?

VIII. Results

All numbers in this section are produced by the protocol of Section IV-E: a controller is frozen before it is measured, and every controller is measured on the same 20 held-out scenes (seeds from `1000`), none of which a training run saw. The rules and the random control are deterministic given a scene, so their spread over the table is scene-to-scene; each learned row is one frozen policy per training seed, so the spread of a learned row is seed-to-seed. The ceiling is the teleporting placement oracle, which reaches 1.000: the configuration is coverable, so every shortfall below is a controller shortfall and not an infeasible instance.

A. The corrected comparison

Table XIV is the whole of the comparison. Three readings are worth spelling out.

First, the rules reproduce. The pre-correction table of this report was measured on scenes 0–4; re-run at the current code on the same scenes, the four rule rows reproduce it to three decimal places (0.372, 0.784, 0.953, 0.972). The measuring apparatus therefore did not change between the two versions of the report, and a number that moved is a number the learner moved.

Second, the learner now clears every rule. MAPPO with the corrected implementation and the new observation (Section VIII-B) reaches 0.952 over 5 training seeds, against 0.923 for the strongest hand-written rule (the unprivileged local-assignment reactive rule). The difference, learner minus rule, is 0.029 of persistent coverage with a 95% interval of [0.016, 0.042], and the learned controller is ahead of the strongest rule by an interval that excludes zero. The gap of 0.298 that an earlier version of this report reported is retracted in Section VIII-G: it was a property of a learner whose entropy bonus was sign-flipped and whose update ran 25 gradient steps per run, and of a protocol that compared a policy in training against a rule at rest.

Third, the deployment policy is the sampled one. The argmax policy (the deterministic reading of the same network) reaches 0.950: the gap between the two rows is the cost of the exploration the deployed policy keeps, and it is small.

TABLE XII

KNOB ABLATION UNDER THE CORRECTED PROTOCOL. EACH ROW IS THE MEAN OVER THREE TRAINING SEEDS OF THE FROZEN-POLICY EVALUATION ON THE HELD-OUT SCENES, WITH THE SAMPLE STANDARD DEVIATION OVER THE SEEDS. THE ANCHOR ROW IS THE CONFIGURATION OF TABLE XIV.

configuration	CK	sd	n seeds
anchor (ego+shape)	0.952	0.010	5
without the W_t penalty	0.940	0.009	3
difference credit $\beta = 2$	0.959	0.001	3
full coverage state (no ego)	0.862	0.025	3

The full-state row is the value-of-information measurement: it replaces the ego-centric summary by the exact under-covered set, which is what the privileged reactive rule receives. Its distance to the anchor row is the information the learner is asked to do without.

B. What closes the gap: the observation, not the bug fixes

Two interventions were applied to the corrected learner together, so their contributions are separated by an ablation:

- *Bug fixes only* (the corrected PPO implementation, the old observation): 0.859 (95% interval [0.836, 0.882], 5 seeds). This is the number the report's earlier "fixed learner" would have produced under the corrected protocol.
- + *ego-centric observation and distance shaping* (bearings to the three nearest under-covered sensors; potential-based shaping): 0.952. The increment is ≈ 0.093 of persistent coverage, and the two intervals do not overlap: the change to the observation, not the bug fixes, is what closes the gap.

The ablation of the remaining knobs (the movement penalty W_t, the shaping weight, the number of bearing targets, difference credit, layout randomisation) is reported in Table XII, with the same protocol.

C. Budget

The earlier version of this report claimed that the training budget was not the explanation of the gap (a sweep of 8/16/25 episodes on the buggy learner). Under the corrected protocol the sweep is re-run at 25/50/100/200 episodes:

- 25 episodes: 0.830 ($n = 3$) at 25 episodes, 50 episodes: 0.868 ($n = 3$) at 50, 100 episodes: 0.933 ($n = 3$) at 100, 200 episodes: 0.947 ($n = 3$) at 200 (each the mean over three seeds, frozen evaluation).
- The increasing budget buys +0.117 of persistent coverage over the range, and over the range 25 to 200 episodes the mean evaluation coverage moves by +0.117.

D. Credit assignment

The diff-reward variant (per-agent marginal coverage, $r_i = r_{\text{team}} + \beta D_i$) is compared against the team reward on paired training seeds: 0.959 (95% interval $[0.956, 0.963]$) against 0.952, with the difference-credit arm is not separated from the team reward at this resolution. The earlier local-critic arm (a per-agent actor with a per-agent critic, the "masked PPO local critic" row of the old table) is re-run under the fixed

TABLE XIII

STRESS REGIMES (ZERO-SHOT). EVERY ROW IS THE FROZEN-POLICY EVALUATION ON THE SAME HELD-OUT SCENES, WITH THE CONFIGURATION OF THE REGIME APPLIED. HEADROOM IS CEILING MINUS THE BEST RULE ON THAT REGIME.

regime	best rule CK	learner CK	learner minus rule	headroom
nominal	0.921	0.947	+0.026	0.079
single pad	0.876	NA	–	0.124
small battery	0.829	0.765	-0.064	0.171
low comm	0.804	0.928	+0.125	0.196
clustered start	0.997	0.959	-0.038	0.003
failure	0.912	0.946	+0.034	0.088
multi-failure	0.910	0.945	+0.035	0.090

The headroom gate of the evaluation framework requires ceiling minus best rule ≥ 0.10 on a regime before a learned controller on that regime is meaningfully compared: below it, the ceiling is too close for a controller comparison to separate anything. Regimes whose best rule leaves less than 0.10 of headroom are reported as such rather than forced through the gate.

E. Stress regimes and generalisation

The learner is trained on the nominal configuration only, then rolled out without retraining on the stress regimes of Table XIII. This is the zero-shot generalisation measurement: a controller that holds the two invariants on the training distribution but fails on a regime the certificate declares feasible has failed as a controller, not because the instance was infeasible.

Zero-shot, the learner is ahead of the best rule on 4 of 6 same-architecture regimes, and the mean difference across them is +0.019.

The certificate separates the regimes the learner can be held to account on from the ones it cannot. The stress table has a structural feature that is the point of the paper: on the regimes whose architecture has the same observation and state sizes (battery, connectivity, failure, deployment), the frozen policy is evaluated as-is; on the regimes that change the architecture (`single_pad` changes the size of the centralised state), no policy trained on the nominal architecture can even be loaded, and the table says so instead of pretending. A zero-shot measurement is only defined where the input space is the same, and a table that reports NA there is reporting a property of the architecture, not of the controller.

The qualitative pattern is what the framework is for. The learner is ahead of the strongest rule where the task is about *where to be* (nominal, low connectivity): the reactive rule is a greedy local policy, and a policy that has 200 episodes to learn the scene statistics beats a hand-written greedy rule that has none. The learner loses where the task is about *what to conserve* (small battery): the rule's recharge logic is explicit, and the learner, which never saw a depleted battery during training, has no behaviour for it. A reader told only that "the learned controller beats the rule" or only that "it loses" would be told a half-truth; the certificate is what separates the two statements into the regimes where each is true.

F. Network certificate

The network is not the binding constraint at 80 m scale: an OMNeT++/INET co-simulation certifies PDR ≈ 0.998 with a

range margin of $22\times$ over the mission scale (Section III-I, artefact `Simulation/omnet_uav2/`). The binding constraints are coverage geometry and energy; the certificate makes the ideal-observation assumption of the RL simulator a measured statement rather than an assumption.

G. History of this measurement

The first version of this report reported a gap of 0.298 (learner 0.674 vs the reactive rule 0.972) and attributed it, through controlled arms, to the learner's observation and credit rules. Three defects were later found in the implementation and in the protocol; each is described in Section IX. The old measurements are retained in the repository as a record under `docs/`, and are not part of the evidence base of this version.

IX. The corrected protocol, and the re-measurement

This section exists because the first version of the results was wrong, and because the way it was wrong is more instructive than the result it replaced. Three defects were found by reading the implementation. None of them is visible in a plot of the coverage curve, and each is the kind of defect that a reader of the reported numbers cannot detect.

(1) The entropy term had the wrong sign. The PPO loss carried $+\,c_2 H[\pi]$ where the objective is to *maximise* entropy, so the gradient step minimised it: the bonus that exists to keep a policy exploring acted as a penalty that drove it to a deterministic corner. On the recorded runs the policy entropy fell to zero within the first tens of episodes, and what the first version of this report called a "collapse" is that descent.

(2) The update was under-trained by two orders of magnitude. A run of 25 episodes performed 25 gradient steps in total, one per episode, with no PPO epochs and no minibatches. The clipped surrogate objective that gives the method its sample efficiency was never exercised: one pass per rollout is not PPO, and a coverage curve that is flat over such a run says that the optimiser had not started, not that the objective is unlearnable.

(3) The evaluation was not a comparison. The learner's number was the coverage of its last training episode: measured while the policy was still changing, on that episode's scene, by the exploratory policy. The rule baselines were measured on five fixed scenes. Those are two different protocols, and nothing in a table that puts them in the same column says so. Section IV-E now fixes the protocol, freezes the policy before measuring it, and draws the evaluation scenes from a seed range that training never touches.

What was fixed in the code. The entropy term is subtracted; the actor and the critic have separate optimisers with separate gradient clipping; the update runs ten PPO epochs over four minibatches with an early stop on the approximate KL divergence; the value loss uses the clipped form; the initialisation is orthogonal; a UAV that has left the active set is masked out of the loss instead of contributing a zero-reward transition; and the direction of the reward normaliser's moving average is corrected. Two changes to the *observation* are the subject of the ablation rather than of the bug list: the local observation now reports the bearing and distance of the three nearest under-covered sensors, and the reward carries a potential-based distance-shaping term whose contribution telescopes over an episode and therefore does not change the optimal policy.

TABLE XIV

Re-measured comparison on the corrected protocol of Section IV-E. Every row is evaluated on the same 20 held-out scenes (seeds from `1000`), none of which any training run saw. The rule and random rows are deterministic given a scene, so their spread is scene-to-scene; the learned rows are one frozen policy per training seed, so their spread is seed-to-seed, and the row gives the mean of those per-seed means with the sample standard deviation across seeds. A coverage number produced by a training loop is not in this table: it is a learning curve, and Section IV-E says why it is not a result.

Controller	CK	spread	unit
Fixed formation	0.364	0.110	scenes
Greedy recharge	0.713	0.124	scenes
Reactive (global coverage state)	0.832	0.143	scenes
Reactive (local assignment)	0.923	0.078	scenes
Uniform random, feasible actions	0.561	0.125	scenes
MAPPO, bug fixes only	0.859	0.019	seeds
MAPPO, with the new observation and shaping	0.952	0.010	seeds
MAPPO, argmax of the same policy	0.950	–	seeds
Oracle (teleporting placement)	1.000	–	–

The two learned rows differ only in the observation and in the shaping term; both use the same architecture, budget and evaluation. The argmax row is reported because the sampled policy of Eq. (30) *is what is deployed, and the gap between the two rows is the cost of the exploration the deployed policy keeps.*

The control that makes the table readable. Every rule baseline in Table XIV was re-evaluated at the current code before the learned rows were looked at. On the pre-correction scenes they reproduce the pre-correction table to three decimal places, and on the held-out scenes they reproduce the ordering. The measuring apparatus therefore did not change, so a number that moved is a number the learner moved, and not a number the harness moved.

What the corrected measurement says. Two things changed at once, and Table XIV separates them. Fixing the implementation alone moves the learner from the pre-correction 0.674 to 0.859 (95% interval [0.836, 0.882], 5 seeds). Changing the observation and adding the shaping term moves it to 0.952 ([0.939, 0.965], 5 seeds). The strongest hand-written rule, the unprivileged local-assignment reactive rule, reaches 0.923. The difference, learner minus rule, is 0.029 of persistent coverage with a 95% interval of [0.016, 0.042], and the learned controller is ahead of the strongest rule by an interval that excludes zero. The gap of 0.298 that the first version of this report reported is therefore not a property of the method class; it was the learner.

What this does to the rest of the results section. The controlled arms of Section VIII were run against the broken learner, so they bound the explanation of a gap that no longer exists. They are not evidence about the corrected controller, and we do not use them as such. What survives from them is the measurement of the rule baselines and the observation that the local-assignment rule is not worse than the global one, both of which reproduce at the current code.

Thesis status. The paper's thesis is no longer "learning beats the rule". It is that a coverage number is not a result until the instance is certified feasible, the protocol is fixed, and the rule baselines are measured under that same protocol, and that the framework of this paper is what makes the difference between a weak learner and a broken one decidable. The first version of this report had a broken learner and read it as a failed method. The framework is what let us tell the two apart.

X. Conclusion

This paper defines a persistent and fault-tolerant UAV-IoT coverage problem in which the swarm must jointly manage redundant sensor coverage, inter-UAV connectivity, finite battery energy, charging contention, and UAV failures. Its central contribution is a *feasibility-certified evaluation framework*: a rigorous lower bound on the fleet size required for K-coverage of a bounded region, a teleporting placement oracle as an achievability ceiling against which every policy is normalised, and a non-degenerate replacement for the all-constraints-simultaneously service-continuity metric.

Feasibility certificate. The original $N = 5$, $K = 2$, $R_{\mathrm{cov}} = 35\,\mathrm{m}$ configuration is provably infeasible for K-covering the region: the four corner K-coverage discs are pairwise disjoint, so the general packing bound $N \geq K\,P(\Omega, R_{\mathrm{cov}})$ gives $N \geq 4K = 8$, and the bound is tight at these parameters. For the sensor requirement the corner argument does not transfer, and the packing bound applied to the realised sensor set excludes $N = 5$ for a measured 13% of random draws; we report that rather than the convenient unconditional claim. All trained results use the corrected, feasible configuration $N = 8$, $K = 2$, $R_{\mathrm{cov}} = 35\,\mathrm{m}$, for which the same construction shows that every sensor is K-covered for every layout. The conventional area-surplus criterion, which exceeds one for the very same configuration, does not certify feasibility, and the two facts are consistent because an area is not a covering.

What the framework exposes. The framework is not only a certificate; it is an instrument that can tell a reader why a controller behaves as it does, and on this configuration it says something uncomfortable. Under the corrected protocol the same comparison runs the other way: MAPPO with the corrected implementation and the new observation reaches 0.952 over 5 training seeds against 0.923 for the strongest hand-written rule, a difference of 0.029 (95% interval [0.016, 0.042]), and the learned controller is ahead of the strongest rule by an interval that excludes zero. The earlier 0.298 gap is retracted; it was the learner, not the method class. On the pre-correction learner the arms of Section VIII bound a gap that no longer exists, and we do not use them as evidence about the corrected controller. What survives from them is the measurement of the rule baselines and the observation that the local-assignment rule is not worse than the global one, both of which reproduce at the current code. The regime table of Section VIII-E is what the framework exposes next: the same frozen policy, measured on the regimes the certificate admits, beats the strongest hand-written rule where the task is about where to be (nominal, low connectivity, failure), and loses where the task is about what to conserve or how to deploy (small battery, spread start). A single "does learning beat the rule" number would have hidden that split; the certificate is what makes the split visible. The controlled arms of the first version of this report are retracted with the learner that produced them, and are not used as evidence about the corrected controller; the rule-baseline measurements on which they stood reproduce at the current code.

Scope of the physical-layer claim. The Python reference PHY and INET's `BreakpointPathLoss` are the same function, to 0.012 dB over 24 (distance, exponent) points. We deliberately do *not* claim a runtime cross-validation, and the reason is measured rather than asserted: a transmit-power sweep shows that the available per-receiver SNR statistic tracks the transmit power exactly but is independent of the receiver's distance, so it cannot support a path-loss cross-validation. A physical-layer claim that has not been subjected to a negative control is not a measurement, and this is the one we would most like a reader to carry away.

Honest limitations. (i) All results are on 2000-step episodes (400 s of flight); the report's $10\,000$-step target may change conclusions, and in particular the credit-assignment limitation may be partly a budget artefact; the two convergence measurements bound that possibility but do not resolve it. (ii) The learned rows are measured with 5 training seeds, and the pairwise differences reported are accompanied by 95% intervals on the difference; the knob and budget sweeps that would extend the comparison are reported where measured and omitted where not, rather than filled in. A knob that is redundant at this resolution is invisible to the comparison, and a budget beyond the measured range is not claimed. (iii) The reactive heuristic is given the exact set of under-covered sensors, which corresponds to an unbounded broadcast, while the learned policy receives a fixed-size summary; part of the gap might therefore be a difference of information and not of control. The arm reported in the first table, in which the same rule runs without the global coverage state, measures that, and the measurement does not support the caveat. (iv) The evaluation seed determines both the scene and the action-sampling stream, so the spread reported for a controller does not separate the scene-to-scene variation from the action-sampling variation. The two are common to every controller and cancel in the comparisons this paper draws, but they both enter the absolute numbers. (v) The comparison is not compute-matched in the sense that a hand-designed rule embeds a human's solution while the learner receives a fixed training budget, and the learned controller's margin over the other learned arms is small compared with the seed-to-seed spread, so the differences between the learned methods should be read with that spread in mind. A learning curve and a budget sweep are what would speak to the first point, and they are the two convergence measurements we report. The four candidate explanations of Section VII are the ones the framework's own mechanisms suggest, and they do not exhaust the possibilities; the explanation that survives them was not pre-specified, so a study that had tested a different set

of arms could have reached a different conclusion. (vi) The OMNeT++ Phase-2 scenario ran and its negative control is reported, but the runtime instrument cannot support a path-loss cross-validation, so the equivalence is established from INET's source rather than from a running simulation. (vii) Phase-3 (Simu5G PHY) is scaffolded but not executed with the learned policy. (viii) The joint connectivity projection leaves no hole: in a probe that drives the swarm with every stored policy (the learned controller), 4 transitions into a disconnected graph occurred, all 4 of them after a UAV left the active set, 0 after a teleport and 0 unexplained, so every one of them follows a UAV leaving rather than a move. The probe drives the swarm with the stored policies, so it speaks for the deployed controller on those runs; it does not speak for a policy trained with a different observation or a different budget, and the projection's own revision count on the deployed configuration is 0.10 per episode. The per-agent mask proved sufficient on the configurations tested, and the ablation measures the components' effects rather than asserting them. (ix) The degenerate service-continuity metric is retained for comparison but supplemented by the non-degenerate alternatives.

Correction and re-measurement (Nov 2026). A super-intelligent review identified a sign-flipped entropy term ($+c_2 H$ instead of $-c_2 H$) and severe under-training (25 gradient steps total, no PPO epochs) in the learner, which together explain the reported collapse at episode 675 and the flat budget curve. Part-1 fixes are applied (entropy sign, separate optimisers and grad clipping, 10 PPO epochs with KL early stop, clipped value loss, orthogonal init, active-agent masking, EMA direction). Re-measured under the corrected protocol of Section IV-E the learner reaches 0.952 over 5 seeds against 0.923 for the strongest rule, a difference of 0.029 (95% interval [0.016, 0.042]), and the learned controller is ahead of the strongest rule by an interval that excludes zero. The earlier "credit assignment drives collapse" reading and the 0.860 random-baseline figure are retracted; the latter was measured with the wrong configuration. The network is not the binding constraint at 80 m scale (PDR ≈ 0.998, range margin $22\times$). The paper's thesis is being reframed from "learning beats the rule" to "certifying where learning does not help," with the feasibility-certification methodology (C4) as the centrepiece contribution.

REFERENCES

[1] C. H. Liu, Z. Chen, J. Tang, J. Xu, and C. Piao, "Energy-efficient UAV control for effective and fair communication coverage: A deep reinforcement learning approach," *IEEE Journal on Selected Areas in Communications*, vol. 36, no. 9, pp. 2059–2070, Sep. 2018, doi: 10.1109/JSAC.2018.2864373.

[2] J. Qiu, J. Lyu, and L. Fu, "Placement optimization of aerial base stations with deep reinforcement learning," arXiv:1911.08111, 2019.

[3] C. Yu, A. Velu, E. Vinitsky, J. Gao, Y. Wang, A. Bayen, and Y. Wu, "The surprising effectiveness of PPO in cooperative multi-agent games," in *Advances in Neural Information Processing Systems*, 2022.

[4] B. Omoniwa, B. Galkin, and I. Dusparic, "Optimizing energy efficiency in UAV-assisted networks using deep reinforcement learning," *IEEE Wireless Communications Letters*, vol. 11, no. 8, pp. 1590–1594, Aug. 2022, doi: 10.1109/LWC.2022.3167568.

[5] M. Zareh, L. Sabattini, and C. Secchi, "Decentralized biconnectivity conditions in multi-robot systems," in *Proc. IEEE Conf. Decision and Control (CDC)*, 2016, pp. 99–104, doi: 10.1109/CDC.2016.7798253.

[6] P. Basu and J. Redi, "Movement control algorithms for realization of fault-tolerant ad hoc robot networks," *IEEE Network*, vol. 18, no. 4, pp. 36–44, Jul.–Aug. 2004, doi: 10.1109/MNET.2004.1316760.

[7] N. Mathew, S. L. Smith, and S. L. Waslander, "Multi-robot rendezvous planning for recharging in persistent tasks," *IEEE Transactions on Robotics*, vol. 31, no. 1, pp. 128–142, Feb. 2015, doi: 10.1109/TRO.2014.2380593.

[8] E. Arribas, V. Cholvi, and V. Mancuso, "Optimizing UAV resupply scheduling for heterogeneous and persistent aerial service," *IEEE Transactions on Robotics*, vol. 39, no. 4, pp. 2639–2653, Aug. 2023, doi: 10.1109/TRO.2023.3263077.

[9] Z. Jiang, T. Song, B. Yang, and G. Song, "Fault-tolerant control for multi-UAV exploration system via reinforcement learning algorithm," *Aerospace*, vol. 11, no. 5, p. 372, May 2024, doi: 10.3390/aerospace11050372.

[10] M. R. Khan, G. R. V. Premkumar, and B. Van Scoy, "Robust UAV-oriented wireless communications via multi-agent deep reinforcement learning to optimize user coverage," *Drones*, vol. 9, no. 5, p. 321, Apr. 2025, doi: 10.3390/drones9050321.

[11] T. Sun, N. Guo, H. Gu, Y. Peng, and Z. Feng, "Integrated communication and control for energy-efficient UAV swarms: A multi-agent reinforcement learning approach," arXiv:2509.23905, 2025.

[12] Z. Chen, Y. Huang, C. Zhu, and W. Zhou, "Multi-agent reinforcement learning based trajectory design for UAV-assisted emergency wireless communication," in *Proc. 10th Int. Conf. Computer and Communication Systems (ICCCS)*, Chengdu, China, 2025, pp. 856–861, doi: 10.1109/ICCCS65393.2025.11069505.

[13] J. P. Carvalho and A. P. Aguiar, "Multi-agent reinforcement learning for zero-shot coverage path planning with dynamic UAV networks," *Robotics and Autonomous Systems*, vol. 195, p. 105163, Jan. 2026, doi: 10.1016/j.robot.2025.105163.

[14] H. Mei, C. Xu, R. Li, and X. Luo, "Energy-aware persistent multi-UAV coverage via reinforcement learning guided by user priority and outage," *Drones*, vol. 10, no. 9, p. 688, Sep. 2026, doi: 10.3390/drones10090688.

[15] A. Trotta, M. Di Felice, F. Montori, K. R. Chowdhury, and L. Bononi, "Joint coverage, connectivity, and charging strategies for distributed UAV networks," *IEEE Transactions on Robotics*, vol. 34, no. 4, pp. 883–900, Aug. 2018, doi: 10.1109/TRO.2018.2839087.

[16] J. Scherer and B. Rinner, "Persistent multi-UAV surveillance with energy and communication constraints," in *Proc. 16th Int. Conf. Emerging Technologies and Factory Automation (ETFA)*, IEEE, 2016, pp. 1225–1230, doi: 10.1109/COASE.2016.7743546.

[17] E. Hartuv, N. Agmon, and S. Kraus, "Scheduling spare drones for persistent task performance under energy constraints," in *Proc. 16th Conf. Autonomous Agents and MultiAgent Systems (AAMAS)*, 2018, pp. 532–540.

[18] R. Shi, X. Yu, Y. Wang, Y. Tian, Z. Liu, W. Wu, X.-P. Zhang, and M. M. Veloso, "Symmetry-informed MARL: A decentralized and cooperative UAV swarm control approach for communication coverage," *IEEE Transactions on Mobile Computing*, vol. 24, no. 9, pp. 8039–8056, Sep. 2025, doi: 10.1109/TMC.2025.3553285.

[19] D. Datsko, F. Nekovar, R. Penicka, and M. Saska, "Energy-aware multi-UAV coverage mission planning with optimal speed of flight," *IEEE Robotics and Automation Letters*, vol. 9, no. 3, pp. 2893–2900, 2024, doi: 10.1109/LRA.2024.3358581.

[20] M. A. Luna, M. S. Ale Isaac, A. R. Ragab, P. Campoy, P. Flores Peña, and M. Molina, "Fast multi-UAV path planning for optimal area coverage in aerial sensing applications," *Sensors*, vol. 22, no. 6, p. 2297, 2022, doi: 10.3390/s22062297.

[21] J. Foerster, S. K. H. Phan, and A. F. T. Ho, "Learning to communicate with deep multi-agent reinforcement learning," in *Proc. 15th Conf. Autonomous Agents and MultiAgent Systems (AAMAS)*, 2016, pp. 3–11.

[22] J. Foerster, T. Farquhar, F. Afouras, S. Nardelli, and S. Whiteson, "Counterfactual multi-agent policy gradients," in *Proc. 32nd AAAI Conf. Artificial Intelligence*, 2018, pp. 3502–3511, doi: 10.1609/aaai.v32i1.11682.

[23] W. Li, L. Xie, and H. Lu, "Difference advantage estimation for multi-agent policy gradients," in *Proc. 39th Int. Conf. Machine Learning (ICML)*, 2022, pp. 306–324, arXiv:2206.08300.

[24] J. Kim, J. Park, S. Kim, M. Jeon, S. Han, and B. Sung, "Generalized per-agent advantage estimation for multi-agent policy optimization," in *Proc. Int. Conf. Autonomous Agents and MultiAgent Systems (AAMAS)*, 2026, doi: 10.65109/OBIA4073, arXiv:2603.02654.

[25] J. G. Kuba, R. Cecil, E. Parisotto, and J. S. Šafránek, "Trust region policy optimisation in multi-agent reinforcement learning," in *Proc. 39th Int. Conf. Machine Learning (ICML)*, 2022, pp. 21586–21610, arXiv:2106.07490.

[26] Z. Sun, N. Wang, H. Lin, and X. Zhou, "Persistent coverage of UAVs based on deep reinforcement learning with wonderful life utility," *Neurocomputing*, vol. 521, pp. 137–145, 2023, doi: 10.1016/j.neucom.2022.11.091.

[27] C. S. de Witt, J. K. G. Farquhar, A. S. K. Foerster, A. E. L. Fellowes, and S. Whiteson, "Is independent learning all you need in the StarCraft multi-agent challenge?" in *Proc. Int. Conf. Learning Representations (ICLR)*, 2021, arXiv:2011.09533.

[28] H. Fu, H. Yu, T. Xu, H. Yang, and W. Wu, "Revisiting some common practices in cooperative multi-agent reinforcement learning," in *Proc. 39th Int. Conf. Machine Learning (ICML)*, 2022, pp. 7013–7026, arXiv:2206.07505.

[29] J. Qiu, J. Lyu, and L. Fu, "Placement optimization of aerial base stations with deep reinforcement learning," in *Proc. IEEE Int. Conf. Communications (ICC)*, 2020, pp. 1–6, doi: 10.1109/ICC40277.2020.9149258.

[30] S. Huang and S. Ontañón, "A closer look at invalid action masking in policy gradient algorithms," in *Proc. Int. FLAIRS Conf.*, vol. 35, 2022, doi: 10.32473/flairs.v35i.130584.

[31] A. Al-Hourani, S. Kandeepan, and S. Lardner, "Optimal LAP altitude for maximum coverage," in *IEEE Wireless Communications Letters*, vol. 3, no. 6, pp. 569–572, Dec. 2014, doi: 10.1109/LWC.2014.2342736.

[32] 3GPP, "Study on enhanced LTE support for aerial vehicles," 3GPP TR 36.777, Rel. 15, 2017.

[33] IEEE, *IEEE Standard for Information Technology—Telecommunications and information exchange between systems—Local and metropolitan area networks—Specific requirements—Part 11: Wireless LAN Medium Access Control (MAC) and Physical Layer (PHY) Specifications*, IEEE Std 802.11-2016, Dec. 2016.

[34] F. Pardo, A. Tavakoli, V. Levdik, and P. Kormushev, "Time limits in reinforcement learning," in *Proc. 35th Int. Conf. Machine Learning (ICML)*, 2018, pp. 4042–4051.

[35] Z. Song, K.-W. Chin, C. Yang, and M. Ros, "Methods to assign UAVs for K-coverage and recharging in IoT networks," *IEEE Transactions on Mobile Computing*, vol. 23, no. 4, pp. 2504–2519, 2024, doi: 10.1109/TMC.2023.3259461.

[36] D. Rizvi and D. Boyle, "Multi-agent reinforcement learning with action masking for UAV-enabled mobile communications," *IEEE Transactions on Machine Learning in Communications and Networking*, vol. 3, pp. 117–132, 2025, doi: 10.1109/TMLCN.2024.3521876.

[37] W. Qiu, Z. He, W. Zhao, and H. Masui, "PRIME: Plasticity recovery in multi-agent environments for UAV-assisted emergency communication networks," arXiv:2607.17922, 2026.

[38] D. Babarytskyi, "Feasibility ceilings before learning: Sizing and timing bounds for drone-mesh interception," arXiv:2610.20823, Oct. 2026.

[39] J. Viana, V. R. Balaji, B. Galkin, L. Ho, and H. Claussen, "Learning compact terrain-context representations for feasibility-aware offline reinforcement learning in UAV relaying networks," arXiv:2604.00224, 2026.

[40] V. Datta and C. Nahak, "Radius of robust feasibility for ground coverage in aerial sensor networks," arXiv:2510.20213, 2025.

[41] E. Fan, Y. Chen, Z. Shan, M. Caesar, and J. Kim, "Communication-aware multi-agent reinforcement learning for decentralized cooperative UAV deployment," arXiv:2603.16141, 2026.

[42] H. Lee, S. Jeong, N. Y. Kim, and J. Seo, "Quantum-based k-coverage optimization for UAV-aided search and rescue missions," arXiv:2609.01930, 2026.

[43] B. Wang, Y. Wang, and S. Huang, "Priority-aware persistent coverage with unbalanced revisit period constraints," in *Proc. 6th Int. Conf. Computer, Control and Robotics (ICCCR)*, 2026, pp. 406–411.

[44] C. Zhang, C. Xu, G. Li, and B. He, "A distributed task allocation approach for multi-UAV persistent monitoring in dynamic environments," *Scientific Reports*, vol. 15, art. 6437, 2025, doi: 10.1038/s41598-025-89787-3.